University of Leicester

School of Computing and Mathematical Science

Engineering Dynamic Succinct Data Structures

Abdulhakeem Ibrahim

Supervisor: Rajeev Raman

A thesis submitted in partial fulfilment of the University's requirements

for the degree of Doctor of Philosophy

July, 2024

Acknowledgements

In the name of Allah, the Beneficent, the Most Merciful. All praises be to Allah who gave me the strength, wisdom, and guidance to complete this thesis.

This has been a long *six-year* journey, and there are many people I would like to thank. First, I would like to thank my PhD supervisor, Rajeev Raman, for patiently supporting and teaching me over these six years. His mentorship and guidance have allowed me to develop both personally and professionally in ways I never imagined.

A huge thank you to my co-supervisor, Dr. Stanley Fung, for his guidance and invaluable support during this journey. I would also like to thank Thomas Erlebach and Fer-Jan de Vries for their support during my GTA duties and panicky moments. To my fellow PhD researchers in the School of Computing and Mathematical Sciences, thank you for sharing time with me and providing helpful advice during our time together.

I would also like to thank the University of Leicester for the Graduate Teaching Assistantship, which supported my research. Likewise, a huge thank you to the Petroleum Technology Development Fund (PTDF) for the financial support provided to allow me to focus on my research. I am also grateful to the Federal University Birnin Kebbi for the study fellowship approved for me.

Last but not least, I would like to thank my parents, brothers, and sisters for their support and constant prayers. My profound gratitude to my wife for her prayers, support, understanding, and enormous patience with me during this journey.

Abstract

When dealing with big data, where storage and query speed are paramount, traditional data structures are fast but have a huge memory footprint. Succinct structures use space close to the information-theoretic lower bound while remaining competitive in query times compared to traditional ones. Achieving space efficiency while supporting dynamic operations efficiently presents a significant challenge. A common naive approach is to rebuild the static data structure for each update, which becomes very expensive as the data structure grows.

In this work, we introduce several innovative dynamic data structures tailored for integer compression, addressing both the need for compact representation and the ability to handle updates and queries efficiently. Our contributions include extensible arrays designed for building append-only Elias-Fano representations that support incremental data additions without requiring complete reconstruction, shown to be competitive with existing solutions.

We also provide a practical implementation of the work of Pătraşcu and Thorup [50] on constant time predecessor search in small sets, which was initially seen as theoretical with no practical implementation. We demonstrate that our implementation is not only simple but also competitive in performance. Leveraging this implementation, we propose a practical implementation of a dynamic searchable prefix-sum data structure based on the works of Dietz [21] and Raman [52]. We show that these theoretical data structures can now be practically implemented with SIMD instructions widely available on recent architectures.

Contents

1 Introduction **1**

1.1 Overview 1

1.2 Motivation 2

1.3 Contributions 3

1.4 Thesis Organisation 3

2 Background and tools **5**

2.1 Introduction 5

2.2 Basic Concepts 6

2.2.1 Model of Computation 7

2.2.2 Memory Architecture 8

2.2.3 Information Theoretic Lower Bound (ITLB) 10

2.2.4 Empirical Entropy 11

2.2.5 Bit-vectors 12

2.3 Space-efficient Data structures 15

2.4 Fixed and Variable-length Codes 16

2.5 Integer Compressors 16

2.5.1 Unary and Binary Codes 17

2.5.2 Elias-Gamma (γ)codes 17

2.5.3 Elias-Delta (δ) Codes 18

2.5.4 VByte Codes 18
2.5.5 Golomb-Rice Codes 18
2.5.6 Packed Encoding 19
2.5.7 Elias-Fano Encoding 19
2.6 Inverted Index Compression 20

3 Extensible Array Representation 22

3.1 Extensible Array 22
3.1.1 Vector EA 23
3.1.2 Simple EA 23
3.1.3 Brodnik et al EA 24
3.1.4 Collection of Extensible Arrays 25
3.2 Our Data Structure 26
3.3 Dynamic Data Structures 29
3.3.1 Append-Only Bitvector 30
3.3.2 Elias-Fano 32
3.3.3 Analysis 34

4 Fast Predecessor Search in Small Set 36

4.1 Introduction 36
4.2 Preliminaries 37
4.2.1 Advance Bit Manipulation Instructions 38
4.2.2 Blind search 38
4.2.3 Significant Bit Positions 39
4.2.4 Compressed keys 40
4.2.5 Ajtai's method 40
4.2.6 Fredman's method 41
4.2.7 SIMD and Builtin Instructions 41

4.3 Computing Significant bit positions 43
4.3.1 Random datasets 44
4.3.2 B-tree sub-tree size datasets 44
4.3.3 B-tree splitter keys datasets 44
4.4 Optimisations 45
4.5 Implementations using built-in instructions 46
4.5.1 Ajtai's method 46
4.5.2 Fredman's method 47
4.5.3 Improved Fredman 48
4.5.4 Experiments 49
4.6 Conclusion 50

5 Dynamic Searchable Prefix Sum 51

5.1 Preliminaries 51
5.2 Related work 52
5.3 Searchable Prefix Sum 54
5.4 Dynamic Searchable Prefix Sum 55
5.5 Operations 56
5.6 Optimisations 57

6 Conclusions 58

List of Figures

3.1 Consecutive blocks of 2^i where $0 \leq i \leq \log n$ 24

4.1 Blind trie for $S = \{7, 85, 102, 200, 205, 232\}$ showing the significant bit positions in the internal nodes and significant bit positions on edges. The leaves hold the index i of $S[i]$ 39

4.2 Number of significant bit positions for various datasets 45

4.3 Time in seconds for 2^{27} *rank* operations on various implementations 49

5.1 Balanced binary tree storing prefix sums of A 53

List of Tables

2.1 All possible bit-strings of length 3 10

2.2 All possible sequences of length 3 that adds up to 5 11

2.3 Documents with their set of terms 20

2.4 Inverted list of terms with the occurrences as document id 21

3.1 Append 24 to the vector and resize 23

3.2 Table showing the worst-case internal fragmentation for various resize factors in a bit-vector 28

4.1 Some of the instructions we used and their description 42

4.2 Result of shifting input of various array sizes by values between $0 - 12000$. . 46

5.1 Array A stored plainly to support *increment/decrement* in $O(1)$ 52

5.2 Array B such that $B[i] = sum(i)$ to support *sum* in $O(1)$ 52

5.3 Array A with original sequence 56

5.4 Array B with possibly out of *sync* prefix sums 56

5.5 SIMD Array C with all the recent updates 56

5.6 Array B after $m = 12$ updates is now refreshed with C 56

Chapter 1

Introduction

1.1 Overview

Big data refers to the massive volume of structured and unstructured data generated at unprecedented rates, necessitating innovative approaches for storage and processing. Digital devices, social media platforms, and IoT (Internet of Things) sensors contribute significantly to this exponential data growth. Estimates suggest that the global data volume could reach 175 zettabytes by 2025, driven by the increasing digitisation of various sectors including healthcare, finance, security, and manufacturing [18]. This rapid data expansion poses significant challenges for traditional storage solutions, which are often inadequate in terms of capacity and cost-efficiency. Consequently, there is a pressing need for advanced storage technologies such as distributed file systems and cloud storage, which offer scalability and flexibility. Equally important is the development of efficient processing algorithms capable of handling large-scale data analytics and processing. Technologies like Apache Hadoop, Apache Spark, and parallel processing systems are critical in managing and extracting insights from big data. As the volume and complexity of data continue to grow, ongoing advancements in both storage and processing are essential to harness their full potential in

driving innovation and decision-making across various fields. The rate at which data grows necessitates an out-of-box approach to efficient storage and query processing.

Traditional processing methods and algorithms are fast when dealing with such datasets but have high space usage requirements. One of the key challenges is to design data structures that are both space-efficient and time-efficient, meaning they use the least space possible while query operations remain fast. The most common approach is to compress the data by exploiting some of its properties, such as repetitiveness. Pre-processing also helps in speeding up subsequent queries (such as search queries) to the dataset. Compressing large datasets reduces space usage but may slow down queries due to the need to partially or fully decompress the data.

1.2 Motivation

Recent research focuses on ways to efficiently compress data and still query the data without having to fully decompress it. This carries many advantages: (i) using less space, (ii) fitting more data in memory, and (iii) executing queries on the compressed data. Succinct data structures [38] are a specific class of data structures that aim to solve this problem by using minimal space while still providing fast query times. These structures are particularly useful in applications where memory is at a premium, such as in DNA or protein sequencing, search engines, and Wikipedia [47]. Using information theory and data compression, the aim is to provide data structures that can fit into primary memory for processing. It becomes harder when the data is dynamic, expecting some updates (insertions and/or deletions), as the folklore approach of rebuilding the data structure is not optimal.

Many algorithms for dynamic succinct data structures have been proposed in the past that provides promising theoretical lower bounds, but some are either impractical or too complex to implement. Their impracticality may be due to lack of available hardware/software technologies to implement them. With rapid development in hardware and software domain,

these could change. In this thesis, we considered, implemented and improved some of these algorithms, and provided optimisations to some.

1.3 Contributions

We propose a range of dynamic data structures that provide improvements upon existing approaches and implementations. These include Extensible Arrays that can be used in many applications, such as the Extensible Elias-Fano data structure. We have also provided a detailed analysis of our experiments. More specifically, our contributions are as follows:

- We present a simple and efficient algorithm for constructing Extensible Arrays (EA). We implement different approaches to the data structure along with some heuristics that optimise them with excellent performance in space and speed. We utilise this to build an Extensible Elias-Fano structure that shows promising improvements over other implementations [47].
- We engineered an implementation of fast predecessor search data structure in small sets with practical implementations. This implementation was built using Single-Instruction-Multiple-Data (SIMD) instructions to achieve parallelism. It is simple, practical, and achieves competitive speed and space usage. To our knowledge, this is the first practical implementation of the work of Ajtai [2] on small static integer sets.
- We provide an efficient implementation of a dynamic searchable prefix sum data structure with *increment and search*. This is built on the earlier data structures we developed and is competitive in speed and space usage.

1.4 Thesis Organisation

The rest of the thesis is organised as follows: Chapter 2 introduces the needed background of the thesis and succinct data structures. Chapter 3, 4, and 5 articulate our contributions,

and finally, Chapter 6 is our conclusion, open problems, and future research directions.

Dynamic Elias-Fano representation: In Chapter 3, we present various implementations of Extensible Arrays (EA). We consider the problem of resizing arrays and propose a more efficient approach. We then utilise the new EA to build efficient Dynamic Elias-Fano implementations for various problem types. Our experimental solutions show that given different scenarios, our approaches are consistently faster than current methods.

Fast Predecessor Search in Small Set: In Chapter 4, we consider the problem of efficient predecessor search in small sets. We provide an implementation of the famous [2] paper on searching small sets that was perceived to be impractical due to the lack of hardware to support some of the theoretical claims. Using SIMD instructions, we provide the first implementation of constant time predecessor search on static integer sets. Our experimental evaluations show that these data structures can be implemented and are also useful, for example, in B-Trees.

Dynamic Searchable Prefix Sum: In Chapter 5, we consider the practical implementation of dynamic searchable prefix sums with *select*. We showed how this data structure can be implemented according to Dietz [21]. We then propose some optimisation methods that could potentially improve the speed of the queries.

Conclusion: Chapter 6 is the conclusion. Key findings and contributions are discussed here, along with future work and open research problems.

Chapter 2

Background and tools

This chapter introduces some key concepts and notation on design and querying of compressed data structures. Starting with the basic concepts, bit-vector which is a key building block of succinct data structures was introduced. Some examples showing how data can be represented using close to the information theoretic minimum was shown and then discussed some well-known techniques and algorithms for compressing integers.

2.1 Introduction

Let us consider a problem where recording the performances of students in a test is necessary. Whenever a student completes the test, their score is computed and stored in a file. For simplicity, we assume that we record only the score of the students as they are ready.

$$3, 50, 7, 28, \ldots, 10, 92, 43, \ldots 76, 43, 29.$$

Now that the results have been collected, it will be useful to be able to analyse the query performances. Let's assume we would like to determine how many students failed the test (where the pass mark is at least 40). It is clear that we have to scan the entire list to get accurate answer. Given $n = 1000000$ be the number of students that took the test, this

means 1000000 scans at best and worst case scenario. This is true for even larger number of students with n number of scans needed to provide answer for such a simple problem.

Pre-prossessing that data in some way can improve on our earlier solution. Let us try with sorting the data in a particular order; ascending order of the scores. With the data sorted, our earlier approach remains theoretically the same (same number of scans at worst case), but can perform better empirically (may not scan through always).

$$3, 7, 10, 28, \ldots, 43, 50, 76 \ldots 87, 91, 98.$$

Even better, solving the problem using divide-and-conquer algorithms such as the infamous binary search makes it easier. Solving the problem becomes incredibly easier using binary search. We only need to find the position of the first occurrence of 40 and we have our answer. This takes about $\log n = \log(1000000) \approx 20$ scans which is ≈ 50000 times faster than our previous approach. This shows that some pre-processing (in this case, ordering the data) can greatly improve some solutions.

Let's take a quick look at storing the dataset. Assuming we naively assign the same number of bits to each score as a 32-bit or 64-bit integer which is equivalent to a *machine word* (discussed in section 2.2.2), this will need $n \log n$ bits to store the data. But we could do better by focusing on the frequencies of appearance of each score. This way, scores with higher frequencies will be assigned shorter codes, and those with lower frequencies of appearance can be assigned longer codes. It is clear that the number of bits needed to represent the dataset is $< n \log n$. More discussions on succinct storage is provided in section 2.2.3 and section 2.2.4.

2.2 Basic Concepts

In this section, we describe the various models of computation that will be used throughout the thesis. We also introduce the concept of memory architecture and management, entropy,

static and dynamic data structures.

2.2.1 Model of Computation

A model of computation is a mathematical abstraction that describes how a computer algorithm or program operates. There are several different models of computation such as Turing machine, the lambda calculus, and the Random Access Machine (RAM). RAM model is the most common model of computation used in analysis of algorithms.

The Turing machine introduced by Alan Turing is a theoretical machine that is often considered the most powerful model. It operates on a tape with symbols, reading, writing, and moving its head according to a set of rules. Despite its simplicity, it can theoretically simulate any other model of computation.

Lambda Calculus focuses on functions and their application. Instead of manipulating symbols on a tape, it uses expressions to define functions and apply them to arguments. It's a powerful way to reason about logic and computation, forming the foundation of functional programming languages.

The Random Access Machine (RAM) is a model of computation that is based on the concept of a random-access memory. The assumption of this model of computation is that a word of w bits can be manipulated with a single instruction at constant time. Typically, w could be 32 or 64 bits, but there are architectures that supports larger words of 128, 256 or 512 bits. Operations such as addition, subtraction, multiplication, division, *mod* ceiling and floors, e.t.c are considered as single operations that can be done in $O(1)$ time. Likewise, logic operation such as bitwise *or*, *and*, *xor*, *not* can be done in constant time. These assumptions do not differentiate access time to the instructions due to memory hierarchy as it is generally proven that access to data in lower memory hierarchy is slower than those in higher hierarchy [31]. But in practice, accessing memory on hardware is logarithmic in

complexity due to virtual to physical address translation [40].

Under the RAM model (which our analysis will be based on), the number of words is used to compute the space usage as well as time complexity depending on the number of read/write operations of an algorithm. One needs to be careful not to make unrealistic assumptions on the RAM model, for example, a sort operation over a set can be considered as an word-level instruction that can sort a set in constant time [16].

2.2.2 Memory Architecture

To assist with understating the problems associated with representing and processing data efficiently, we describe the memory architecture of computer system. The hierarchical structure of the computer memory provides balance between speed, capacity, and cost. It comprises multiple levels, each serving distinct roles in data storage and access with higher level of the hierarchy having less space but more speed and the lower level having more space and slower. Typically, data can be stored in the following types of memory [23]:

- Registers: At the top of the memory hierarchy are registers, which are small, fast storage locations within the CPU. Registers hold data that the CPU is currently processing. Their limited size and proximity to the CPU ensure minimal delay in data access, facilitating the execution of instructions at high speed.
- Cache Memory: Cache memory acts as an intermediary between the CPU and the main memory (RAM). It is designed to temporarily store frequently accessed data and instructions, thereby reducing the average time required to access memory. The cache is divided into multiple levels:
  - L1 Cache: Located within the CPU, this is the smallest and fastest cache.
  - L2 Cache: Slightly larger and slower, it is usually situated on the CPU chip.
  - L3 Cache: Even larger and slower, L3 cache is shared among multiple CPU cores

in modern multi-core processors.

The cache leverages spatial and temporal locality to predict and store data that is likely to be used soon, significantly speeding up data retrieval processes.

- Main Memory (RAM) RAM, or Random-Access Memory, is the primary working memory in a computer. It stores data and instructions that the CPU needs while performing tasks. RAM is much faster than disk storage but slower and more expensive per byte than cache memory. It connects to the CPU via a memory bus, which facilitates the transfer of data. Despite its speed, RAM is volatile, meaning it loses all stored data when the computer is turned off.

- Secondary Storage: Secondary storage, such as hard disks and solid-state drives (SSDs), provides non-volatile, long-term data storage. This level of memory is much larger and cheaper per byte than RAM, but significantly slower. Data in secondary storage must be loaded into RAM before the CPU can access it. SSDs, while faster than traditional hard disks, still do not match the speed of RAM or cache.

- Tertiary and Off-line Storage: At the lowest levels of the hierarchy are tertiary and off-line storage solutions, including optical disks, magnetic tapes, and cloud storage. These provide large-scale, cost-effective storage for data that is infrequently accessed. While they offer high capacity, their access speeds are much slower, making them suitable for backup and archival purposes rather than regular data retrieval.

In summary, programs that efficiently utilise fast memory execute much quicker. Specifically, succinct data structures aims to fit more data into the faster cache or RAM, hence minimising the need for time-consuming data transfers from slower secondary storage. This leads to quicker data access and improved overall performance.

2.2.3 Information Theoretic Lower Bound (ITLB)

When dealing with a combinatorial objects, it is quite important to understand the number of bits needed to optimally represent each object. If we know that we are to represent an object drawn from a set S, the minimum amount of bits needed to uniquely represent and identify the object is $\lceil \log |S| \rceil$ bits in the worst case where each object is given codes of same length.. This is referred to the *information theoretic lower-bound*. Any attempt to use less bits to represent the object other than its information theoretic lower bound may not support unique decodability of elements of S. This means more than one object from S may be represented in the same way. However, if we know more about the characteristics of the elements in S, we may be able to use less than the ITLB.

For example, if S is a set of bit-vectors all having length n. This means that $|S| = 2^n$ which represents all the bit-vectors of length n, and $\lceil \log 2^n \rceil = n$ which matches the ITLB. However, if the assumption is that S contains all bit-vectors with m of their bits set to 1 or 0, then $|S| = \binom{n}{m}$. The ITLB now is $\lceil \log \binom{n}{n} \rceil \approx n \log \frac{n}{m} + O(n) bits$ by using Newton's coefficient formula and Stirling's factorial approximation [54]. We will show different data structures and their information-theoretic lower bounds.

Bit-vector: when representing a bit-vector of length n, we have shown that $|S| = \log 2^n$ possible bit-strings in S with the length of n. For example, there are 2^3 possible bit-strings of length 3, and they are

000	001	010	011	100	101	110	111

Table 2.1: All possible bit-strings of length 3

Proposition 1: The Information theoretic lower bound to represent a bit string of length n is $\lceil \log 2^n \rceil = n$ bits.

Remark: At worst case, writing out a bit-string as it is represents the ITLB.

Binary trees: A binary tree is rooted tree with each node having a maximum of 2 children; left and right child or none. There are $C_n = \frac{1}{n+1}\binom{2n}{n}$ possible variations of binary trees where n is the number of nodes. For $n = 3$, we now have $C_3 = \frac{1}{3+1}\frac{6!}{3!.3!} = 5$

Proposition 2: The information theoretic lower bound to represent a binary tree with n nodes is $2n - O(\log n)$

Prefix sum: Given a set containing sequences of n increasing positive integers that sums up to m with the assumption that m and n are known in advance, we can show that the variations of these sequences are $|S| = \binom{m-1}{n-1}$. Using an example, if $n = 3$ and $m = 5$, we have $\binom{7!}{2!.3!} = 6$

Proposition 2: The information theoretic lower bound to represent prefix sum is $\lceil \log |S| \rceil \leq n \log(\frac{m}{n}) + n \log e$ bits.

[1,2,5]	[1,3,5]	[1,4,5]	[2,3,5]	[2,4,5]	[3,4,5]

Table 2.2: All possible sequences of length 3 that adds up to 5

2.2.4 Empirical Entropy

We introduced the concept of Information theoretic lower bound that tells us the least number of bits needed to represent an item from a sequence in the worst case scenario. This means that we do not have any prior knowledge on the data we intend to store, hence we do that analysis at worst-case. But if we do have some idea on what we will be storing, it is easier to explore and exploit some properties of the data such that we can use less than the Information theoretic lower bound while still maintaining unique decodability.

Entropy is the lower bound of the number of bits needed to uniquely represent an object in a set. Given the problem of encoding an a set of elements A over the alphabet $\sum =$

$\{a_1, a_2, \ldots a_n\}$ with the occurrence of any $a_i \in A$ represented as $|a_i|$. The probability of a_i in A is given by $p(i) = |a_i|/|A|$. The $0th$ order empirical entropy of A is $H_o(A) = \sum p(i) \log 1/p(i)$. This is called the $zero-order$ or $memory-less$ entropy because the object a_i emitted by the source is independent of all previously emitted objects $a_{i-1}, a_{i0-2}, \ldots a_0$. Let's give an example here with bit-string of size n. If the bit-string has 50% of $1s$, then the $H_0 = 0.5 \log 0.5 + 0.5 \log 0.5 = 1$. Here, we need exactly n bits to represent the object. In another example with the bit-string having 70% of $1s$, then $H_0 = 0.7 \log 0.7 + 0.3 \log 0.3 = 0.8816$ bits. We can now see that by simply knowing the frequencies of the alphabets, we can use lower than the ITLB to represent the bit string using $0.8816n$ bits. only.

It gets more interesting if the source emitting these symbols has some *memory*, where the probability of a symbol to be emitted depends on the appearance of that symbol in the set. This is called the $higher - order$ entropy or $k^{th} - order$ entropy [44]. The probability of a_i given than $a_0 \ldots a_{n-1}$ has been emitted is given by $p(a_i) = p(a_i|a_0 \ldots a_{n-1})$. Hence, the $k^{th} - order$ entropy $H[a_0 \ldots a_{n-1}] = \sum p(a_0 \ldots a_{n-1}) H[a_0 \ldots a_{n-1}]$.

2.2.5 Bit-vectors

Bit-vectors are very important building blocks of succinct data structures representing sets, strings, and sequences. A bit-vector is often referred to as bitmap or bit-array in various literature. Henceforth, we assume a bit-vector to be a sequence of bits that supports *access*, *rank* and *select* operations.

Definition: Bit vector is a sequence $B[0, n-1] \in 0, 1^n$ of length n that supports the following operations:

- $access(B, i)$: returns the bit $B[i]$ for $0 \leq i \leq n-1$.
- $rank_1(B, i)$: returns the number of occurrence of $1s$ in $B[0, i)$. This can be written as $\sum_{j=0}^{n-1} B[j]$ for $0 \leq i \leq n-1$.

- $rank_0(B, i)$: returns $i - rank_1(i)$ for $0 \leq i \leq n-1$.
- $select_1(B, i)$: returns the position of the ith occurrence of 1 in B. This can be written as $max\{j|rank_1(i) = i\}$.
- $select_0(B, i)$: returns the position of the ith occurrence of 0 in B. This can be written as $max\{j|rank_0(j) = i\}$.

Naive Approach

By storing the bit-vector in plain format as sequence of n bits, *access* operation can be supported in *constant* time. Using additional look-up table T_r such that $T_r[i]$ stores the number of $1s$ till i will allow us to support *rank* operations in constant time. But T_r takes additional $n \log n$ space. To support *select* in constant time, we also use look-up tables T_{s0} and T_{s1} the stores the positions of all $1s$ and $0s$ respectively. Clearly, this approach is time efficient as it supports all operations in constant time, but it is not *space-efficient* as it needs $n + O(n \log n)$ bits of space to achieve this.

Example

- *bit-vector*: 1011001110001001
- $n = 16$
- T_r: the look-up table to support $rank_1$ operation

1	1	2	3	3	3	4	5	6	6	6	6	7	7	7	8
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

- T_{s1}: the look-up table to support $select_1$ operation

1	3	4	7	8	9	13	16
1	2	3	4	5	6	7	8

- T_{s0}: the look-up table to support $select_0$ operation

2	5	6	10	11	12	14	15
1	2	3	4	5	6	7	8

To get $rank_1(6)$, we simply look-up $T_r[6]$ which returns 3. That means there are 3 set bits from the beginning of the bit-string to position 6. For $rank_0(6)$, we simply do $i - rank_1(6) = 3$. To calculate $select_1(6)$, we look-up $T_{s0}[6] = 9$. This means the 6th set bit is at position 9 in the bit-string. For $select_0(6)$, we look-up $T_{s0}[6] = 12$.

We can improve on the above by not storing all the $rank_1[i]$ in T_r, but after every $\log^2 n$ positions. This means T_r occupies $n/\log n$ space. To support $rank_1$, we can read only one entry in T_r and access at most $O(\log n)$ bits. *select* is also supported by doing *binary-search* over the values of *rank*. This takes $n + O(n/\log n)$ space and the operations take $O(\log n)$ time which is better than the naive approach, but not *optimal*.

Succinct Solution

The bit-vector is divided into *sub-blocks* of $\log n$ bits and *super-blocks* of $\log^2 n$ bits. For each *super-block*, we store the number of set-bits of all *sub-blocks* to the *super-block*. As each *super-block* is $\log n$ in size, this required $O(\log n.n/\log^2 n) = o(n)$ bits. For each *sub-block*, we store the partial *rank* that resets at the beginning of each *super-block*. This requires only $\log \log^2 n/\log n = o(n)$ bits [43].

Example

- *bit-vector*: 10110011100010011011101010001011
- $n = 32$
- *super-block* size: $\log n = 5$
- *sub-block* size: $\log^2 n = 25$

For the purpose of this example, we will assume the memory word is now $w = \log n = 5$.

bit-vector	10110	01110	00100	10001	11000	00010	11
super-blocks	0					11	
blocks	0	3	6	7	9	0	1

To answer $rank_1(i)$, we sum the *rank* values in the *super-block* and *sub-block* containing i. As these stores the *global* and *local rank* respectively, we only need to determine the exact number of bits set inside the *local* block up to position i. This can be done in constant time using *mask* and *popcount* on $\log n$ bits (single *sub-block*) or at most $2 \log n$ bits (intersecting *sub-blocks*). Other implementations of of *rank* and *select* can be found in [6, 20, 45, 47, 49]

2.3 Space-efficient Data structures

Space-efficient structures are designed to minimise storage requirements while allowing queries to remain fast. The main challenge for these data structures is to balance between keeping query time as low as traditional data structures, but using space close to *Information-theoretic lower bound*. While most solutions for these data structures are build to be *static*, a lot of research is being done on making them *dynamic* to support update queries.

Succinct data structures were first discussed in Jacobson's Doctoral thesis as a special class of compressed data structures [38]. While compressed data structures may need $O(n)$ additional bits to store an object that requires at least n bits to represent, succinct data structures will only need $o(m)$ additional bits. Unlike traditional data structures that may use standard space storage, compact structures aim to approach the information-theoretic lower bound (ITLB) of space required to store data, otherwise called the *succinct bound*. Trees and text data can be stored compactly using special encoding and application-specific designs. Using a sequence A on n elements as an example, a compact data structure will use $n + O(n)$. On the other hand, succinct data structures are highly-space efficient with the aim of fitting more data into memory to achieve very fast queries (typically constant time). The space usage is typically $n + o(n)$. Theoretically, these bounds are promising as they can be considered

negligible under the asymptotic analysis. However, this may not always be the case in practice and theory may not always translate to practice [32, 31].

2.4 Fixed and Variable-length Codes

Variable-length codes and *fixed-length* codes are two fundamental approaches to encoding data, particularly integers. Fixed-length codes assign a uniform number of bits to each possible symbol in the set, ensuring simplicity and ease of decoding. However, this often results in inefficiency when encoding symbols with varying probabilities of occurrence. In contrast, variable-length codes assign shorter bit sequences to more frequently occurring symbols and longer bit sequences to less frequent ones [44]. This adaptive allocation optimises the overall bit usage, reducing average code length compared to fixed-length codes, especially when dealing with data with varying entropy. Let's look at an example

- *Sequence A*: a b c d
- *fixed-length codes*: a: 000 b:001 c:010 d: 011

Assuming we know that the objects in A have varying frequencies, then we may use variable-length codes to improve on the encoding.

- *Sequence A*: a (60%) b (30%) c (3%) d 2%)
- *variable-length codes*: a: 0 b:1 c:010 d: 011

2.5 Integer Compressors

Integer compressors are designed to compress sequences of integers in a way that minimises storage space while allowing efficient access, retrieval, and sometimes modification of the compressed data. As most of the data structures we designed in this thesis are built on integer compression, we give an overview of some of the integer list compression algorithms.

The main aim to ensure that we assign shortest uniquely-decodable, variable-length codes to each integer so that the sequence is encoded with minimal number of bits.

2.5.1 Unary and Binary Codes

Unary codes represents a using a sequence of *'0's* followed by a single *'1'*. The number of *'0's* indicates the value being encoded. For example, the unary encoding of the number 3 is 0001. Unary encoding is simple and intuitive but becomes inefficient for larger numbers due to its linear growth in length with the encoded value.

$$u(x) = 0^{x-1}1$$

One can also use the reversed unary with $u(x) = 1^{x-1}0$ and encode 3 as 1110. Binary codes $B(x)$ represents numbers using the *base-2* numeral system, where each digit (bit) in the sequence represents a power of 2. In binary encoding, numbers are expressed using only *'0's* and *'1's*. This method is highly efficient for computers because it directly corresponds to the hardware's binary logic. For example, $B(3)$ will be 11.

2.5.2 Elias-Gamma (γ)codes

This method encodes in unary the length of the binary representation of —x— and then encodes x in binary without its highest bit. Hence, $\gamma(x) = u(|x|)[x]_{|x|-1}$ where $[x]_l$ is the least significant bit of x. We can show that $|\gamma(x)| = 2|x| - 1 = O(\log x)$ [25]. Assuming we are encoding numbers that fits into a computer word, γ codes for small integers can easily be decoded in $O(1)$ by either using a small *look-up* table or using machine instructions to find the *highest-bit* of x. For example, $\gamma(39) = 11111000111$ where $B(39) = 100111$ with length of 6 bits, then the $U(6) = 1111110$. Concatenating the *unary* and stripping off the leading 1 from the *binary*, we have 11111000111

2.5.3 Elias-Delta (δ) Codes

While *Elias-γ* codes are efficient for smaller numbers, they can be inefficient for larger integers due to the unary prefix. Elias-Delta coding improves on this by replacing the unary prefix with a *gamma code* of the length of the binary representation. For instance, $\delta(113) = 001111110001$, where the last part 1110001 is the binary representation of 113, prefixed by $\gamma(7) = 00111$. The decoding of delta codes follows naturally from the decoding of gamma codes.

2.5.4 VByte Codes

This method works really well for large numbers. The aim to speed up decoding by reading fixed-aligned data chunks . Here, integers are represented using a multiple of fixed number of bits, this can be *a nibble, byte, or even word.* Variable-byte [55] splits each integer into groups of *7 bits.* Each group is stored in one byte. The most significant bit (MSB) of each byte is used as a continuation bit, indicating whether the byte is the final byte of the current integer (continuation bit = 0) or if there are more bytes to come (continuation bit = 1). This method ensures that smaller integers use fewer bytes, though not very suitable for integers that needs less than a byte. For example, $VB(913) = 1110010\underline{1}00000001$ with the 8th bit (control bit) set to 1 showing that we need to read the next byte to decode the integer. We also had to pad the 2nd byte with zeros to make it a whole byte.

2.5.5 Golomb-Rice Codes

In the early 1970s, Robert [53] introduced Rice coding, a parameterised compression scheme for integers that is particularly effective when the values are clustered around a common value. Rice coding is a special case of Golomb coding and is defined by a parameter k. The Rice code $R_k(x)$ is composed of two parts: the quotient $q = \lfloor (x-1)/2^k \rfloor$ and the remainder $r = x - q \cdot 2^k - 1$. The quotient is encoded in unary, and the remainder is encoded in binary

using k bits. For example, with $k = 5$, $R_5(113) = 000110000$, where the quotient 3 is encoded as 0001 and the remainder 16 as 00000. Using $k = 6$, the code becomes $R_6(113) = 01110000$, which is more efficient because 2^6 is closer to 113 than 2^5. Decoding involves counting the leading zeros to determine the quotient, then calculating the value by multiplying the quotient by 2^k and adding the remainder.

2.5.6 Packed Encoding

Packed encoding methods improve compression ratio and decoding speed by encoding blocks of integers instead of individual values. Binary packing, for instance, represents a block of integers using the minimum number of bits necessary for the largest integer in the block. Simple strategies like Simple-9 and Simple-16 pack integers into 32 or 64-bit words using a few selector bits to indicate how many integers and their bit sizes are packed. Simple-9 uses 4 selector bits and 28 data bits, allowing configurations like 28 1-bit integers or 14 2-bit integers. Simple-8b uses 64-bit words with 4-bit selectors. The QMX method improves on this by packing integers into 128-bit words and storing selectors separately, compressing them with run-length encoding (RLE) for efficiency [44].

2.5.7 Elias-Fano Encoding

Elias-Fano encoding, proposed independently by Elias[24] and Fano[26] in the 1970s, is effective for compressing sorted integer sequences. Each integer in the sequence is split into low bits and high bits. The low bits are stored explicitly, while the high bits are encoded in a bitvector using unary coding. For example, given a sequence $S(n, u)$, each integer $S[i]$ is divided into low bits ℓ_i and high bits h_i, with the high bits stored in a bitvector such that the i-th position is set to 1 at $h_i + i$. This method efficiently represents both the high and low parts of the integers, balancing compression and decoding speed. More discussion on Elias Fano encoding is given in Section 3.3.2.

2.6 Inverted Index Compression

Search engines like databases, store documents in their repository. An index is maintained to allow for faster queries and retrievals. Let's consider an example with the following web pages (henceforth referred to as documents) containing some terms/words :

- *Doc1* → (book, car, house, money)
- *Doc2* → (cat, money)
- *Doc3* → (car, cat)
- *Doc4* → (book)
- *Doc5* → (money, house)

Table 2.3: Documents with their set of terms

This is very similar to the index at the beginning of books or even this thesis, showing for each page, what terms (topics or sub-topics in this case) can be found in them. In large web systems, query operations across web documents comes with obstacles. The term(s) being queried could occur in very large number of documents with each document having number of terms. The index needs to be optimised in a way that relevant documents to the search terms can be retrieved and returned quickly. Using inverted index, we map terms to their occurrences in a collection of documents, typically represented as a list of document identifiers called *posting list*. However, due to their potentially large size, especially in extensive document collections (like the world-wide-web), it is essential to employ compression techniques to reduce storage requirements and enhance query performance [13, 41]. We show what an inverted index of Table 5.6 will look like below:

- $book \rightarrow (1, 4)$
- $car \rightarrow (1, 3)$
- $house \rightarrow (1, 5)$
- $money \rightarrow (1, 2, 5)$
- $cat \rightarrow (1, 3)$

Table 2.4: Inverted list of terms with the occurrences as document id

Inverted index data structure are very useful in supporting full-text search engines by mapping each term to a list of documents in which it appears to perform rapid searches, retrieve relevant documents, and rank them efficiently based on various algorithms [47]. Likewise in Information Retrieval (IR) systems, inverted indexes are used to quickly locate documents that contain specific words or phrases, making them indispensable for digital libraries, academic databases, and other large text repositories [41]. Moreover, Version Control systems like Git use inverted indexes to manage and retrieve different versions of source code efficiently. By indexing changes and differences between versions, these systems can quickly show the history of changes, compare different versions, and merge updates from multiple contributors.

Given their usefulness and the large amount of data they hold, it is important that they benefit from compressed or even better, succinct data structures. This would mean more data can be loaded in memory for faster processing. Some of these algorithms have been discussed in Section 2.5. Further reading on inverted index compression can be found in [3, 4, 47, 59]

Chapter 3

Extensible Array Representation

This chapter will present existing work that was done in the area of extensible arrays. We will begin by introducing dynamic arrays that supports append-only operations along side the standard static operations. We then focused on extensible arrays of non-standard width ($1 \leq width \leq w$). We then use the dynamic arrays to build an efficient Elias-Fano implementation that supports *append-only* operations. We show variations of this data structure that works well in various settings.

3.1 Extensible Array

Arrays are excellent example of static data structures that have been studied extensively [44]. Vector is the most basic form of extendible array that supports random access in constant time. They have been shown to solve a number of dynamic data structures problem such as randomised queues, stacks, priority queues and queues [11, 44, 12], and have been used in building complex compact data structures [42, 51]. We describe our Extendible Array (EA) which was built upon the idea of vectors. We also show other variants of EAs of non-standard widths (such as bit-vectors, shorts. characters e.t.c). The operations supported are:

- $create(n)$: Create an array of initial size n
- $access(i)$: Access the item at index i.
- $append(x)$: Adds a new item at the end of array.
- $grow()$: resize the array to new size $m > n$

3.1.1 Vector EA

A Vector is a common example of extendible array that supports *access* operation in $O(1)$ time and $grow/shrink$ in amortised constant time using $n + O(n)$. To append a new item to a vector of size n that is already full, we need to increase it's size. The most common approach is to double the size of the vector to $2n$ to allow for up to n new appends before the need to resize the vector. This may create internal fragmentation problem of $n - 1$ of a vector of $2n$. Likewise, using first-fit allocator, a sequence of $grow()/shrink()$ could lead to inefficiencies such that despite the array using n space, the allocated range of memory could span $\Theta(n \log n)$. A typical vector supports only word-size elements, but we are also interested of elements of non-standard width.

Algorithm 1 Grows A by n

```
procedure GROW()
A_o is new array of size 2n
for i := 1 to size(A) do
    A_o[i] := A[i]
end for
return A_o
```

Example

14	56	32	

14	56	32	24				

Table 3.1: Append 24 to the vector and resize

3.1.2 Simple EA

By choosing a fixed resize factor f, we can reduce the internal fragmentation from above. Normally, f is a power of 2. This keeps the $O(1)$ access and append time and worst case

$O(n)$ amortised time for grow operation. We improve the worst case internal fragmentation to $O(f)$ but significantly degrade speed of grow operation.

3.1.3 Brodnik et al EA

Brodnik [11] proposed a singly extendible array that use $n+O(\sqrt{n})$ space at all times (where n is the total number of elements in the array). They achieve this by dividing the array into consecutive *data−blocks* DB_A of the sizes 2^i where $0 \leq i \leq \log n$. A conceptual *super−block* SB_A of size $2^{\log n}$ contains *data-blocks* of the same size. As each data block is of size $2^{\lceil i/2 \rceil}$, a *super-block* SB_i will have a maximum of $2^{\lfloor i/2 \rfloor}$ data blocks until it is full. Therefore, SB_i will have a total of 2^i elements in it. To further keep space usage low, memory is only allocated for non-empty blocks.

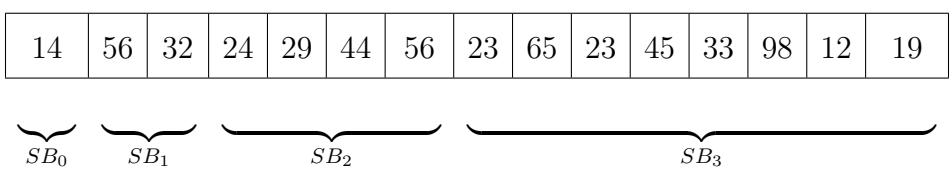

Figure 3.1: Consecutive blocks of 2^i where $0 \leq i \leq \log n$

The algorithm for access operation is shown below. A key aspect of it is how to compute the position of the leading set bit in the binary representation of $i+1$ in constant time. Brodnik [12] gives a method that uses basic arithmetic and logic operators to achieve this in constant time. But modern machines allows for this as well using Bit Scan Reverse instruction [17].

Algorithm 2 Access(i)

1: Let r be the binary representation of $i-1$ with no leading zeros
2: Let the *super-block* $k = \lfloor \log i + 1 \rfloor$.
3: Let $b = \lfloor k/2 \rfloor$ bit of the *super-block* after the leading 1-bit
4: Let e be the remaining $\lfloor k/2 \rfloor$ bits of r.
5: **return** the element e in *data-block* b of *super-block* k.

As all the steps in *access* can be done in constant time if we ignore the cost of dynamic memory allocation, the running time is $O(1)$. Space wasted from internal fragmentation is $O(\sqrt{n}$ words which they show to be optimal.

3.1.4 Collection of Extensible Arrays

Joannou [39] described various dynamic random access data structures built as Collection of Extendible Arrays (CEA), which are essentially, a vector of pointers to other vectors. A vector of size n is divided into consecutive blocks of fixed size $b > 1$, and records are stored in fixed-size data blocks of size b each. A vector of n/b pointers (called the index block) stores a pointer to each of the data-blocks. To perform $access(i)$ operation, we only fetch the $(i \mod b) - th$ entry in the $\lfloor i/b \rfloor - th$ data block. It is clear that access operation is $O(1)$ (though, could be slower due to 2-random accesses), and $O(1)$ amortised constant time for grow operation. This approach clearly reduces the internal fragmentation, though, the choice of b needs to be carefully made to avoid having $\theta(b)$ wasted space per data block.

Another approach is to vary the size of the blocks as done in [11]. The vector of size n is divided into super blocks of size $1, 2, 3 \ldots, 2^{\lfloor \log_2(n) \rfloor}$. And each super block is sub-divided into data blocks of equal sizes. The access operation takes constant time, while grow operations takes $O(1)$ amortised time. The wasted space is $O\sqrt{n}$ words. The modified Brodnik shares same analysis, though handles the data structure a bit differently. All data blocks are of same initial size $b = 2$. A grow operation adds an element to the last data block, and when full, we either double the index or copy two existing data blocks into a newly created data block of size $2b$.

Global Brodnik tries to minimised the internal fragmentation by "self-tunning" the block sizes to an optimal size of $b = \theta(\sqrt{N/t})$ where t is the number of current EAs created and N is the total number of elements. Whenever a block size is twice b, the block is either halved or doubled which leads to the re-organisation of all the EAs in the CEA. Access time is $O(1)$

and grow is amortised constant too. The internal fragmentation is $O(\sqrt{Nt})$.

One thing to note is that CEAs are normally slower in supporting random *access* than conventional Vector representation. This due to making 2 random access to arrays (index and data blocks respectively), while conventional vector needs only one random access. A solution to this is to make the index block as small as possible to fit into the fast memories so that one of the random accesses will at least be very fast. We will explore this later in depth.

Raman [52] proposed two implementations of collection of extendible arrays (CEAs) using $o(n)$ extra space. They considered EAs that are of non-standard widths with first implementation supporting all operations in $O(\log n/\log\log n)$ amortised time while the second supports look up in constant time and updates in $O(n^e)$ time. Their data structure has more functionalities than ours, as it supports *insertion* at any index $i < n$, while we support only *insertion* at the end of A. Raman and Rao [51] proposed an EA with support for elements with various *width*. If r is the width of the elements, they store the EA using $s+O(aw+\sqrt{saw})$ bits of space where n is the number of elements in the EA, $s = n*r$, a is the number of EAs. Joannou [39] also gave an empirical study of the CEAs while emphasising on external fragmentation while we are interested in the representation of one-dimensional array. The need to perform 2 random memory access to perform an *access* which in turn slows down the data structure in practice.

3.2 Our Data Structure

We consider the simple approach of resizing extedible arrays by a fixed factor instead of doubling it all the time. The aim here is to identify the optimal resize factor for vectors of different widths. As such, we did not only consider elements of known word-size (32-bits and 64-bits), we consider extendable arrays of elements that require less than word-size number of bits to be represented (such bit-vectors, characters, shorts e.t.c).

Starting with a vector V initial capacity of $c = O(w)$ (where w is the word-size), we increase the vector when full by an additive factor, say f. The appends are done in bits (as the vectors can be of any width ($1 \leq width \leq w$)). We **append in bits** until the array is full, then we re-size (create an array of bigger size and **copy in words**). We try to exploit this feature and show that we do not have to always double the size (as done conventionally), rather we can increase by a factor less than double with two gains - reduce internal fragmentation while remaining fast. In general, the amortised cost of re-sizing an Array of n bits by $f = k$ bits is

$$\frac{n/w + k/w}{k} = O(\frac{n}{kw}) \tag{1}$$

By taking the value of $k = n/w$,

$$O(\frac{n}{kw}) = O(1) \tag{2}$$

As we increased by $f = O(k/w)$, we use $o(n)$ wasted space at worst-case. Let's give an example with a simple bit-vector with various resize factors f.

n *in bits*	*f=n* (≈ 50%)	*f=n/2* (≈ 25%)	*f=n/4* (≈ 12.5%)	*f=n/8* (≈ 6.25%)	*f=n/16* (≈ 3.13%)	*f=n/32* (≈ 1.56%)	*f=n/w* (≈ 0.78%)
4096	8192	6144	5120	4608	4352	4224	4160
8192	16384	12288	10240	9216	8704	8448	8320
16384	32768	24576	20480	18432	17408	16896	16640
32768	65536	49152	40960	36864	34816	33792	33280
65536	131072	98304	81920	73728	69632	67584	66560
131072	262144	196608	163840	147456	139264	135168	133120
262144	524288	393216	327680	294912	278528	270336	266240

Table 3.2: Table showing the worst-case internal fragmentation for various resize factors in a bit-vector

While we are able to resize V using $o(n)$ extra space and support resize operation in amortised $O(1)$ time, we also need the consider that the frequency of the *resize* operations. They increase with reduced *resize-factor* f. Suppose we want to *resize* V currently having n items by factor f to accommodate m elements, then we need to determine the number of *resize operations* needed.

Example 1: Given an array of $n = 16$ elements, and we want to determine how many *resize* operations are needed before it can accommodate 512 elements with a resize strategy of doubling. Let's denote k to be the number of resizes.

$n.2^k \geq m$

$16.2^k \geq 512$

$2^k \geq 32$

$k \geq 5$ is the minimum number of resize operations needed.

Example 2: For f less than doubling, we proceed as follows:

- Initially, we have $a_0 = n$
- When full, we resize to $a_1 = n + n/f$
- Then again to $a_2 = a_1 + a_1/f$
- ...
- $a_k + 1 = a_k + a_k/f$

By finding the general term of the geometric progression and determining the common ratio, we can get the formula as:

$$a_k = n(\frac{f+1}{f})^k$$

From our earlier example, if we want to determine how many resize operations are needed for $f = 1/4$, then we solve $16(\frac{5}{4})^k = 512$, using logarithms, we solve for k as $k \geq 16$. This means that new need at least 16 resize operations to get 512 elements into the array. Compared with Example 1, that's almost 3 times more resize operations.

3.3 Dynamic Data Structures

We proposed and describe three dynamic data structures; append-only bitvector, append-only Elias-Fano and Dynamic Elias-Fano with special inserts. We have seen that append-only bit-vectors are specialised variants of vectors (with *width=1*). Our implementation reduces internal fragmentation to minimum while remaining fast (append in amortised constant time and access in O(1)). On the other hand, Elias-Fano is able to compress a given set S close to its 0th order empirical entropy only if the $n = |S|$ and universe u are known. But what happens if we either know only u but not n or neither u and n?. We developed a technique to apply Elias-Fano on S even if we do not know the above information. However, this technique will lead to lost of *something* in space and time, and we have shown that this lost is negligible factor in space and time

3.3.1 Append-Only Bitvector

Many succinct data structures are built upon bit-vectors. Given an array of bits A with length n, we support the following operations:

- $access(A, i)$: returns the bit b at position i for any $1 \leq i \leq n$.
- $rank_b(A, i)$: returns the number of occurrences of b from positions $A[1]$ to $A[i]$.
- $select_b(A, i)$: returns the position of the ith occurrence of b in A.

Jacobson [38] and Clark [15] described a design for constant time solution on bit-vector based on blocking technique. This divides the bit-vector into conceptual blocks to reduce the number of bits needed to process to support the operations. Since then, there are numerous papers on uncompressed succinct bitvector representation [43, 56, 58]. We propose a simple data structure that support the above operations efficiently while using succinct space. We add an *append* operation to the existing set of operations.

- $append_b(A)$: appends b to A.

Starting from an empty Bitvector A, append bits to A while supporting fast *access*, *append*, *rank*, *select* operations at any time (even in-between appends) while maintaining succinct space. To our knowledge, this is the first practical implementation of bitvector with append. [33] proposed a bitvector with append data structure based on *RRR-bitvector* and achieved $nH_0(A) + o(n)$, where H_0 is the zero-order empirical entropy of A, but did not provide any practical implementation of their solution. Our approach will use $n + (n/\log n) = n + o(n)$, which comes close to RRR when the ratio of $1s$ to $0s$ is about the same.

Bitvector Layout

The layout of our bitvector will represent A as an array of words. For every bit b in A, b_i is located in word (array index) i/w and position $i \mod w$ in the word. The array will be maintained as an array of fixed size starting from $size = O(w)$ bits, which will be re-sized

by $newsize = n + n/w$ bits when full (where n is the number of bits in the array at the time of resizing). The space to store A having n bits will be $n + o(n)$ at all times. We support *rank* and *select* operations as described in section 2.2.5.

Operations

To perform an $append(A, b)$, we first check if the bitvector A is full and re-size it. Then we identify the block where b will be inserted and append it to that block. We then update the necessary rank samples D and select samples S. This bitvector uses an extra space of $o(n)$ on top of n for the bit-vector at worst case.

To support rank, we divide A into conceptual blocks of fixed size B bits and store the aggregate count of set bits from the beginning of A up-to block B_i in *rank-samples* array $D[i]$. By choosing block size of $B = \log^2 n$, this incur an overhead space of $n \log n / \log n^2 = o(n)$. We start with D having size of $n_d = w$, and resize by $n_d + n_d/f$ its current size, whenever it is full. Hence, to support $rank$, we use $o(n)$ extra space. To answer $rank_1(i)$, we get the number of set bits up to the super-block where i resides by doing an array look-up as $r1 = D[\lfloor i/B \rfloor]$. Then we continue to scan word-wise while aggregating the number of set bits (using popcount) until we exceed position i, and then we re-scan the last word and use masking to get to the right position i, and count the number of set bits to add to the result $r1$. The maximum number of blocks to scan is $\log n$.

We build our select structure on top of the existing $rank$ structure. We divide A into conceptual blocks of variable length with each block containing exactly $s = \log^3$ ones. We store the position of the is^{th} 1 in $S[i]$ for $0 \leq i \leq n_1/s$, where n_1 is the number of set bits in A, as such, S uses $n \log n / \log n^3$. The additional space used to support select is still $o(n)$. To solve $select_1(A, i)$, we look-up $p = S[i/s]$, if i is a factor of s, we return p. Else, we lookup $q = D[p/B]$ to determine the block boundary where p is, and then scan word-wise until the number of $1s$ is greater than i. We then re-scan the last word using *broadword-programming* or advanced *built-in* instructions and find the exact position of the ith 1 in A.

3.3.2 Elias-Fano

Given $S = \{s_1, s_2, s_3, \ldots, s_n\}$ of increasing monotone integer sequence with S upper bounded by some value $u \geq s_n$, we represent each $s_i \in S$ in binary using $\lceil \log u \rceil$ bits. Then we split each of the representation of s_i into an upper part of length $\lceil \log n \rceil$ bits, and the remaining $\lceil \log(u/n) \rceil$ bits to be the lower part. The lower part of each $s_i \in S$ are stored explicitly using $n\lceil \log(u/n) \rceil$ bits. The upper parts are stored as a unary representation of the cardinality of their buckets using a bit-vector of length $2n$. The bitvector should support *rank* and *select* operations to allow for the encoding to access the ith element in S in constant time. Other queries such as *membership*, *nextGEQ*, *predecessor* and *successor* queries can be supported. Therefore, we can store S using $n\lceil \log(\frac{u}{n}) \rceil + 2n$ bits.

$Access(S, i)$ returns the ith element in S in constant time. This is achieved by first getting the bucket where i^{th} bit will fall by counting the $0s$ from beginning to the position of the ith 1, by solving $k = select_1(H, i) - i$. Concatenating k in binary and $L[i]$ gives the desired result. In general, the space usage in bits of Elias-Fano encoding of S with n increasing monotone integers upper bounded by u is given by:

$$EF(S) = n\lceil \log(\frac{u}{n}) \rceil + O(n) \tag{3}$$

To show how well Elias-Fano performs when it comes to compressing sequences, we compare it space usage to that of 0th order empirical entropy of a bitvector $B(n, m) = n \log(m/n) + O(n)$ (where n is the length of the bitvector and m is the number of set bits), it can be seen to be very similar. This means, that a sparse bitvector can be represented compactly as the positions of its set bits, which will give a sequence of increasing monotone integers.

An Example

Suppose $u = 24$, $S = \{2, 3, 5, 7, 11, 13, 24\}$, and $n = 7$, we represent each integer in S using $\lceil \log 24 \rceil = 5$ bits in binary. Each lower bit will take $\lceil \log(24/7) \rceil = 2$ bits, and upper bits

will take $\lceil log(7) \rceil = 3$ bits. The lower bit array will be $L = \{10, 11, 01, 11, 11, 01, 00\}$ using 14 bits, and the higher bits will be $H' = \{000, 000, 001, 001, 010, 011, 110\}$. We now encode H' based on the cardinality of its buckets in unary (empty buckets included, and using 0 as a stop bit). This gives the encoding of H' as $H = 11011010100010$. Let's assume we want to answer $access(S, 3)$. We solve $select_1(H, 3) - 3 = 4 - 3 = 1$. We concatenate 1 with $L[3]$ to get 101 which is the desired result.

Extensible Elias-Fano with *append-only*

Given an empty sequence S of which we know the universe u, but did not know the length of the sequence beforehand. Elements can be added to S in *append-only* fashion. Pibiri [47] described a data structure for that. They maintained a small buffer array B of size m that stores uncompressed integers until full. When B is full, the sequence is now compressed and appended to an array of blocks encoded with Elias-Fano. This uses $O((m + \frac{n}{m} + 1). \log u)) = o(n)$ bits of space using *y-fast* trie for $n = \omega(\log^3 u)$. Access operation is supported in $O(1)$.

Our approach start with an Elias-Fano $EF(S)$ with an unknown universe u. We will always use the current information of u and n to determine the length of lower bits and upper bits. We append into the EF until it is full, then we resize by $O(n/wk)$, and copy the elements from the old EF re-encode with the new information of n into the new EF, then destroy the old EF. This approach is clearly not optimal when $|s|$ becomes large, but internal fragmentation is now $o(n)$ at worst case except when doubling then goes up to $O(n)$. We can reduce the internal fragmentation by resizing with lower factors f and pay slightly on speed degradation. A more optimal approach is based on the property below and the proof can be found here [47]:

Property 1: Given a monotone sequence S where $n = |S|$, let $S[i, j)$ and $S(j, n]$ represents *sub-sequences* of S. For any $i < j < n$, we have

$$EF(S[i, j)) + EF(S(j, n]) \leq EF(S[i, n])$$

We use the idea of Collection of Extensible Arrays (CEA) [39, 51] by dividing S into data blocks of sizes $b_i = 2^i$ where $0 \leq i \leq \log n$. We plan to store S as a collection of "Extendible Elias-Fano". Each data block B_i is an Elias-Fano representation of the block starting with size n_i, and resized by f when full. An array of pointers of size $\log^2 n$ called the index block I will hold pointers to each data block with $I[i]$ pointing to block B_i. The idea is that B_i can hold the Elias-Fano representation of increasing monotone integers with known universe of $2^i - 1$ and unknown length $\leq b_i$.

Access operation will need to first determine the Data block B_i to access by checking the index block, then doing the normal access operation on B_i. As I is just $\log n$, it can fit to faster memory (cache) with very limited impact on the query. We can improve further by δ encoding the Data blocks there by reducing the universe $B_i(u)$ of each data block, hence achieving better compression. A likely draw back is that lower level blocks may have longer sequences to store, and may slow down the resize process. We intend to experiment further on how to improve on this.

3.3.3 Analysis

The space usage for this implementation is simple, we need the space to store the index block I, and then space for Elias-Fano representation of each data block. is as follows:

- Size of *Index* block I is $\log^2 n = o(n)$.
- Data blocks will be $\sum_{j=0}^{\log n} EF(B_j) \leq EF(S)$.

We now describe the operations supported by this data structure

- *access(j)*: to access the jth element, we begin by calculating the *data-block* it will belong. As each block is of size 2^i, the jth element will be in block $\lfloor \log_2 j \rfloor$, and then we can offset the counter to find and return j.
- *append(i)*: to append, we check if the most recent *super-block* is full, then we resize by

$f = o(n)$. And then we append to the first available position in the last *super-block*.

Chapter 4

Fast Predecessor Search in Small Set

In this chapter, we discussed the problem of predecessor search operation in small sets that was perceived to be impractical due to lack of hardware to support some of the theoretical claims. Using SIMD instructions, we provided the practical and efficient implementation of constant time predecessor search of integer set leveraging Single-Instruction-Multiple-Data (SIMD) instructions. Our experimental evaluations shows that these data structures can be implemented and also useful, for example, in B-Trees, and Super-scalar search and sorting.

4.1 Introduction

Predecessor search is amongst the most fundamental algorithmic problem in computer science [19]. An important sub-case is that of searching a small static set of integer keys. On the word RAM with word size w bits, "small" is usually taken to mean $w^{O(1)}$. A number of papers have explored this problem [2, 30, 46] and have proposed $O(1)$-time and linear-space solutions to this problem, improving upon comparison-based approaches that would take time logarithmic in the number of keys ($O(\log w)$ time in this case). These solutions are either described in the *cell probe* model [2], where the algorithm is only charged for the reads and writes it makes to memory (and can compute arbitrary functions on data in registers for

free), or make complicated, and ingenious, *bit-parallel* tricks to simulate SIMD operation on several items packed into w-bit words. Their complexity has hitherto led to these algorithms being viewed as impractical.

In recent years, the range of instructions built into CPUs has increased tremendously, and this leads us to re-examine the practicality of the above approaches. In addition, all of the existing approaches broadly use a *blind trie* search (see below for a description). The practicality of these approaches depends heavily on the number of *significant bit positions* in the input keys (see below, but very roughly, considering the keys as bit-strings, we place them in a trie, and look at which positions are in fact needed to navigate to a leaf). In a set of k keys, the number of significant bit positions in the worst case is $k-1$ and the above algorithms proceed with this assumption. However, in the best case the number of significant bit positions only needs to be $O(\log k)$, and practical applications may or may not have many significant bit positions. For example, in one application we consider, namely maintaining a dynamic bit-vector [49], the keys are the prefix sums of the sizes of the sub-trees of a given node; these behave much better than the worst case. Based upon these insights, we implement and empirically evaluate predecessor search algorithms based on [2] and [30], and show clear improvements over comparison-based methods.

4.2 Preliminaries

Let X be a set of n (where n is $|X|$) be a set of positive integers drawn from universe U. We define the following for any $x \in X$:

- *access(i):* returns the elements $X[i]$ for $0 \leq i < n$.
- *predecessor(x):* returns the maximum element $y \in X$ such that $y \leq x$.
- *member(x):* returns *true* if $x \in X$ and *false* otherwise.

As we are using the *word-RAM* model, arithmetic and logical operations are expected to be

performed in $O(1)$ time. There some additional operations that we will expect to also run in *constant* time such as counting the number of trailing/leading bits in s and number of set bits in s_i. Most common theoretical approach before is to use a pre-computed look-up table, but with advancements in *built-in* instructions [1, 5, 37], we are able to support this practically in $O(1)$.

4.2.1 Advance Bit Manipulation Instructions

We define an operation $lcp(x, y)$ that for two integers x and y that computes the bit position where x and y shares the most common prefix. We use built-in functions and compiler intrinsics to achieve this in constant time by computing the $exclusive - or$ between x and y, and then count the number of leading zeros using LZCNT [1] to determine the longest common prefix.

Algorithm 3 LCP(c, y)

```
1: z =: x xor y
2: return lzcnt(z).
```

4.2.2 Blind search

A Patricia trie $P(S)$ otherwise called a compact trie is derived from a normal trie by replacing each sub-string on an edge with its first character (also called a *branching-character*), and merging any only child with its parent. A $blind - search$ introduced by Ferragina [28] looks for a given string s in S by comparing only the available branching characters. This search procedure ends at either of the following: (a) s is exhausted and a leaf v is reached; or (b) a mismatch happened when branching from a node v. Ferragina [28] suggests that any leaf u in the sub-tree of v shares the *longest common prefix* (LCP) with s. The value $l = LCP(s, S[u])$ indicates the bit position where s should have branched out. With this new information and

[1] https://software.intel.com/sites/landingpage/IntrinsicsGuide/

another blind search for s, the exact position of s in S can be computed.

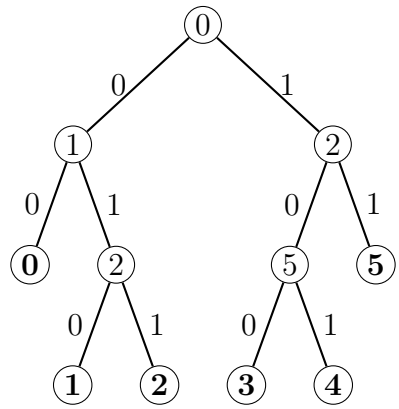

Figure 4.1: Blind trie for $S = \{7, 85, 102, 200, 205, 232\}$ showing the significant bit positions in the internal nodes and significant bit positions on edges. The leaves hold the index i of $S[i]$

4.2.3 Significant Bit Positions

A branching point is the index of the longest common prefix between two integers x, y. Given an input of w-bit integers S of size $k = |S|$, the set of significant bit positions $B(S)$ is all the distinct bit positions of $lcp(s_i, s_{i+1})$ for all $s_i \in S$ and $0 \leq i \leq k-1$. The upper bound on the number of significant bit positions for k keys is k, while the lower bound is $\log k$. This is trivial to see that the upper bound is possible when all the keys have a different *lcp*, while on the lower bound side, we need at least $\log k$ bits to be able to uniquely identify k elements.

Lemma 1. Given a set of sorted w-bits integers S of length k, computing the *lcp* between consecutive adjacent elements in S gives all the significant bit positions in S.

Proof: Two adjacent integers $(x, y) \in S$ shares the longest common bits compared to any other combination of $(x, z) \in S$, which makes the lemma to hold trivially. If (x, z) has an *lcp* greater than (x, y), then $z < y$, and that violates the sorting order of S.

Example 1.

- Let $S = \{7, 85, 102, 200, 205, 232\}$

- $k = 6$
- Set of significant bit positions is $B = \{0, 1, 2, 5\}$ after removing duplicates.
- Mask generated from $B = 228$ which is 11100100 in binary (with padding).

4.2.4 Compressed keys

Fredman et al. [30] proposed the idea of using compressed keys as part of their data structure that supports constant time predecessor search on S with $k = w^{1/6}$. Given a key x, a compressed key $\hat{x}$ is a key containing only the bits of x at significant bit positions (with some additional padded zeros). Key $\hat{x}$ is of length $b \leq k^3$. A set of compressed keys $\hat{S}$ contains the compressed keys from S. We proposed to use some compiler intrinsics to compress x in constant time and be of length $b \leq |B|$. We first create a mask m of length k with the bits at index i in m to 1 if i is a branching bit, and 0 otherwise. Then we use the PEXT[2] to extract the bits at the positions specified by the mask.

Algorithm 4 Compress(x, m)

1: **return** $_pext_u64(x, m)$.

Example 2.

- Let $S = \{7, 85, 102, 200, 205, 232\}$
- The mask generated from significant bit positions of S is $m = 228$
- $\hat{S} = \{1, 5, 7, 12, 13, 14\}$

4.2.5 Ajtai's method

Ajtai [2] presented a constant time data structure for searching in small integer set S. They build a Patricia trie $P(S)$ of the elements of S represented as binary strings of length $\lceil \log u \rceil$

[2]https://software.intel.com/sites/landingpage/IntrinsicsGuide/

where u is the universe of the set. Assuming that $P(S)$ fits into a machine word, they show that they can search for a given key x in $P(S)$ and returns an index i of the leaf matching x in constant time. This is because of the assumption that operations on machine word takes $O(1)$ time. Then they computed $j = lcp(S[i], x)$, and complete the search by another operation that returns the rank of x in $P(S)$ assuming x branches out at j. All these operations are in constant time using RAM model. They concluded that *"the algorithm is not very realistic as searching through a trie requires more than constant time in actuality"*.

4.2.6 Fredman's method

Fredman [30] improved on the above by introducing *rank* on a set in constant time assuming all the elements of the set can fit in machine word w. To do predecessor search on S, they pre-process S to extract all the significant bit positions (with extra irrelevant zeros) and store them in an array B, and then compressed S by extracting only the significant bit positions in every element in S and form another array of compressed keys $\hat{S}$. Assuming $\hat{S}$ also fits into a machine word, they can compute $rank(\hat{S}, i)$ in constant time. They argue that either $y = S[i]$ or $y = S[i+1]$ has the longest common prefix j with x. If the compressed trie was used, then i and $h = rank(B, i)$ determines the edge (u, v) where x should have branched out. The predecessor of x is the largest key below v if $x_j = 1$; otherwise, the successor is the smallest key below v. Using lookup tables to perform these operations, they showed that they can support this in constant time.

4.2.7 SIMD and Builtin Instructions

SIMD instructions allows for parallel processing of k data set per instruction where k is determined by the processor and w. SIMD registers comes in various sizes of 128-bits (in SSE) and 256-bits (AVX/2), and most recent but not widely available 512-bit (AVX512) registers supports larger data set. We consider the 256-bit registers that comes with AVX/2, and are widely available in most *Intel processor* machines.

Instruction	**Type**	**CPUID**	**Description**
$PEXT(a, b)$	Builtin	BMI2	Extracts and deposit bits from a based on positions specified by mask b
$LZCNT(a)$	Builtin	LZCNT	Count the number of leading zeros in a
$TZCNT(a)$	Builtin	BMI1	Count the number of trailing zeros in a
$_mm256_set1_epi16(a)$	SIMD	AVX	Broadcast a as a 16-bit element to all elements of the 256-bits register
$_mm256_loadu_si256(*a)$	SIMD	AVX	Loads and read into a 256 register 256-bits of data specified by memory location a
$_mm256_xor_si256(a, b)$	SIMD	AVX	Horizontally computes the bitwise XOR of 256-bits registers a and b
$_mm256_movemask_epi16(a)$	SIMD	Created	Creates mask from the most significant bit of each $16 - bit$ element in 256-bits register a
$_mm256_minpos_epi16(a)$	SIMD	Created	Horizontally computes the minimum amongst the packed 16-bits elements in 256-bits register a

Table 4.1: Some of the instructions we used and their description

Bit-manipulations have increasingly became important to fast data structures, with most approaches relying on some clever *bit-hacks* and programming tricks [35]. Both the algorithms we considered in this paper are heavily reliant on Bit-wise manipulation to perform its operations. This makes their approach complex and challenging. We exploit the Bit Manipulation Instructions (BMI) that are an extension of x86 instruction set architecture that operates on general purpose registers. We used instructions that can be largely grouped into the following:

- *Load and store*: this group of instructions load and copy data from continuous chunk of memory into a SIMD register. When the data to be read is from a non-continuous memory locations, this slows down these operations as you will need a separate *read* operation for each location.

- *Element-wise*: this group of instructions performs an operation on a specific data

element within a single or multiple SIMD registers. These could include $shifting$ and $access$.

- *Horizontal-wise*: these sets of instructions act across the elements of a SIMD register. It allows for operations to be performed on values store within a register. For example, adding or multiplying two values within the same register and permutation of values within a register.
- *Advanced BMIs*: these are bit manipulation instructions implemented as single instructions in *intel* processors that were considered expensive before. These includes $PDEP$, and $PEXT$.

4.3 Computing Significant bit positions

We show an experiment to compute the number of branching positions in various data sets which are all synthetic. Our first dataset is aimed at generating the worst case number of significant bit positions, the second dataset generated a uniformly distributed random numbers using the C++ STL random number generator[3], the third dataset is based on the sizes of B-tree nodes, and the last dataset is based on the b-tree splitter keys.

Worst Case: We show a potential worst-case input scenario that could generate the maximum possible significant bit positions. Let's assume we want to generate k integers, we generate a sequence S of length k as powers of 2 by setting $S[i] = 2^i$ for $0 < i < k$.

Example 3.

- Let $k = 6$ and word size $w = 5$ bits.
- Then $S = \{0, 2, 4, 8, 16, 32\}$
- The set of branching position $B = \{0, 1, 2, 3, 4, 5\}$

[3]https://en.cppreference.com/w/cpp/numeric/random

4.3.1 Random datasets

For the random dataset, we used the C++ Mersenne Twister pseudo-random generator engine[4] to generate small sets of 64-bit integers. We then sort the resulting set and compute the number of significant bit positions.

4.3.2 B-tree sub-tree size datasets

Given k, we set b (the BTree fan-out) to be equal to k, and h (the height of the BTree) be a random number. We now initialise S with initial size of say $n \leq k$ and each $S[i] = b^h$ for $0 \leq i \leq n$. We now make b^h random increments in S with two conditions: (a) if $S[j] < 2b^h$, then set $S[j] = S[j] + 1$ (b) else if $S[j] = 2b^h$, we split $S[j]$ into two. We set $S[j] = b^h$, then shift $S[j + q]$ to the right where $1 \leq q \leq n$, and finally, $S[j + 1] = b^h$, and the increment continues. When we finish the increments, we then get the first k elements in S, and take a prefix sum of their values as our dataset.

The idea here is that we simulate BTree sub-tree sizes of fan-out b and height h. This means the size of the B-tree cannot be greater the b^h, and whenever insertions leads a node j to be greater than b^h, we split the node into two sub-tree.

4.3.3 B-tree splitter keys datasets

The splitter keys in a b-tree are the largest/smallest keys the sub-trees. We create a b-tree sub-tree dataset D of size N where N is large and take its prefix sum. Given k to be the size of the required input, we now set $S[i] = D[i * k]$ for $0 \leq i < k$ to get the smallest splitter keys of the b-tree. We can also set $S[i] = D[i * (k + 1)]$ for $1 \leq i \leq k$ to get the largest splitter keys of the b-tree.

[4] http://www.cplusplus.com/reference/random/mt19937/

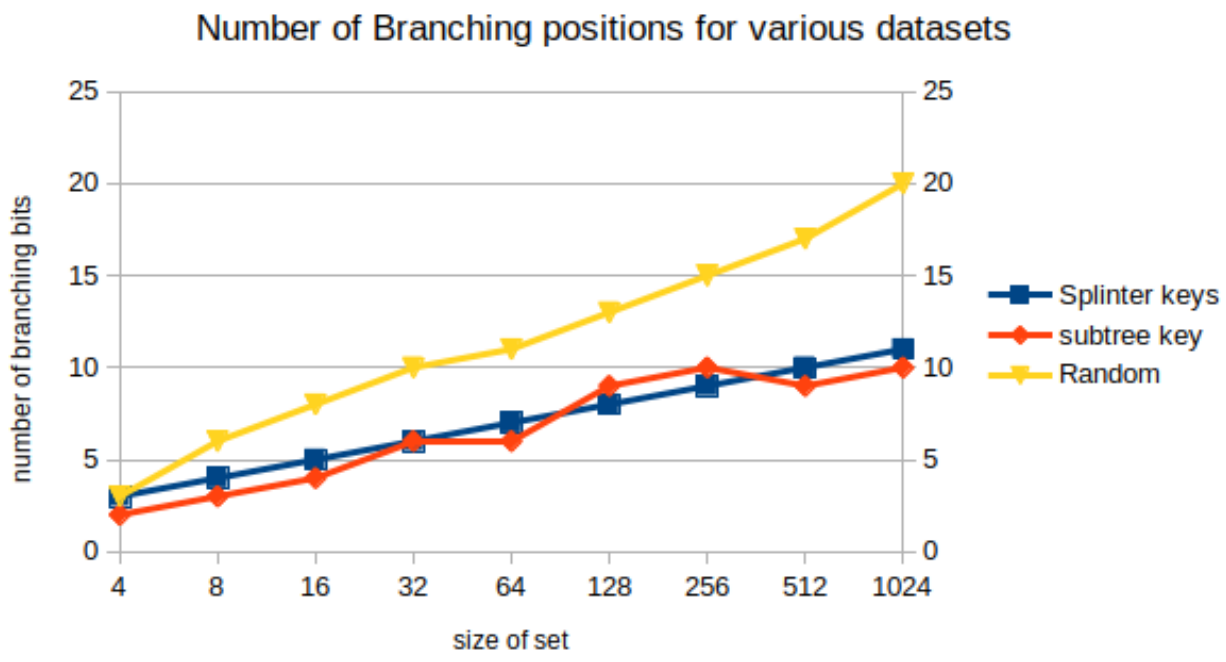

Figure 4.2: Number of significant bit positions for various datasets

4.4 Optimisations

We propose a technique to reduce the number of significant bit positions in a set S. We hypothesise that if we $shift$ the set S to $S + x$ by making $S[i] = S[i] + x$ for all $1 \leq i \leq k$, where x is a random integer, the significant bit positions of some values of S will be shifted, which will in turn increase the number of duplicates in B and in turn reducing the number of significant bit positions. We generated random inputs of various sizes and shifted each input by values i where $1 \leq i \leq 2^{16}$. We computed the maximum, minimum and average number of significant bit positions for each of the data sets report them below. The baseline is the number of significant bit positions of the generated input before the shifting is started.

Metric/Input Size	**4**	**8**	**16**	**32**	**64**	**128**	**256**	**512**	**1024**	**2048**
Baseline	3	5	6	9	9	11	12	13	13	15
Minimum	2	3	4	4	6	7	8	9	10	12
Average	3	5	7	8	9	10	11	12	14	15
Maximum	3	7	11	12	14	14	15	16	19	19

Table 4.2: Result of shifting input of various array sizes by values between $0 - 12000$.

From table 4.2, we can see there is some improvements in the number of significant bit positions at some shift value which can reduce space usage of Ajtai [2] lookup tables. We intend to explore this even more as part of our future work. We intend to answer questions such as: *what value of i do we shift S to reduce the number of significant bit positions below the baseline?*

4.5 Implementations using built-in instructions

In this section, we will describe how we implemented two data structures that supports fast predecessor search on small set of integers S. We will also discuss their potential strengths and weakness, and propose some improvements that potentially speed up their execution time in practice.

4.5.1 Ajtai's method

To support the blind search of key x in constant time, we propose to use a pre-computed lookup table with all the possible answers called the *Answer-Table* or *AT* in short. Since this is done once during the pre-processing stage, it does not affect the speed of the search. We begin by getting the set of significant bit positions B of size b and generate a mask m from B. We then take all the possible values of length b bits and store in $AT[i]$ the result of blind search $BS(i)$ for $0 \leq i \leq 2^b$. The size of this table is $O(2^b)$. We also generate

a *Correction-Table* or *CT* in short, that takes the result i from the *AT* and the computed $j = lcp(S[i], x)$ and produce the correct position of x in S. Essentially, *CT* is of size $k\lceil logu \rceil$, where u is the universe of S. The predecessor of x can be trivially computed by returning

Algorithm 5 rank(S, x)

```
1: y =: compress(x, m)
2: i =: AT[i]
3: j =: lcp(S[i], x)
4: return CT[i][j]
```

$S[rank(x, S)]$, and the successor of x as $S[rank(x, S) + 1]$. All operations are supported in constant time. The major drawback to this apparoach is the extra space incurred of $O(2^b)$, which is largely dependent on the number of significant bit positions.

4.5.2 Fredman's method

To support rank on w as proposed by [30], we use an AVX 256-bit register that supports up to $4w$-bits. We are able to find the $rank(\hat{S}, \hat{x})$ in constant time using some carefully written instructions described below. We broadcast $\hat{x}$ into 256-bit register a, then read $\hat{S}$ into another 256-bit register b. Since AVX does not support *greater-or-equal* comparison, we compute the *maximum* horizontally between a and b and then compare for equality between the result and b. The predecessor and successor of x can now be computed as done in Ajtai's

Algorithm 6 rankFW(S, x)

```
1: a =: _mm256_set1_epi16(x)
2: b =: _mm256_loadu_si256((__m256i*)S)
3: c =: _mm256_cmpeq_epi16(_mm256_max_epu16(b, a), b)
4: d = _m256mm_movemask_epi16(c)
5: i = _mm_tzcnt_64(d)
6: i = lcp(S[i], x) < lcp(S[i + 1], x)?i + 1 : i
7: j = lcp(S[i], x)
8: return CT[i][j]
```

method. All operations are supported in constant time as well. This is better than [2] in

space usage as it does not need the extract space of lookup table AT. The major drawback to this is the number of operations needed to compute the *rank*. It apparently becomes slower than Ajtai's method.

4.5.3 Improved Fredman

We propose to speed-up Fredman [30] *rank* operation. Since the first part of the search procedure in [30] tries to find the element in $\hat{S}$ with the longest common prefix with $\hat{x}$, we describe our new approach as follows:

- Broadcast $\hat{x}$ into an AVX register a
- Load $\hat{S}$ into an AVX register b
- Let $c = a$ xor b
- Find the index of the minimum element i in c
- $S[i]$ is the element with the longest common prefix with x

This proves to be faster and more efficient than the approach in [30]. It uses less instructions hence improve the speed of the operation while maintaining no extra memory footprint.

Algorithm 7 rankFWImproved(S, x)

1: a =: $_mm256_set1_epi16(x)$
2: b =: $_mm256_loadu_si256((__m256i^*)S)$
3: c =: $_mm256_xor_si256(a, b)$
4: d =: $_mm256_minpos_epu16(c)$
5: i =: $_mm256_extract_epi16(d)$
6: j =: $lcp(S[i], x)$
7: **return** $CT[i][j]$

4.5.4 Experiments

We implemented the aforementioned data structures using C++ and various tests were conducted to study the speed of each of them. We also implemented and compared them with branch-less binary search as described in [7]. The machine used was an Intel i5 64-bit machine clocked at 3.40GHz, 8GB RAM running Ubuntu 22.04.3 LTS. The compiler version is g++ 11.4.0 with maximum optimisation level set. We used the *clock()* method to measure CPU time and reported in seconds. All tests were run 10000 times and the average time was taken. We used random data to do the test as it generates a larger number of significant bit positions.

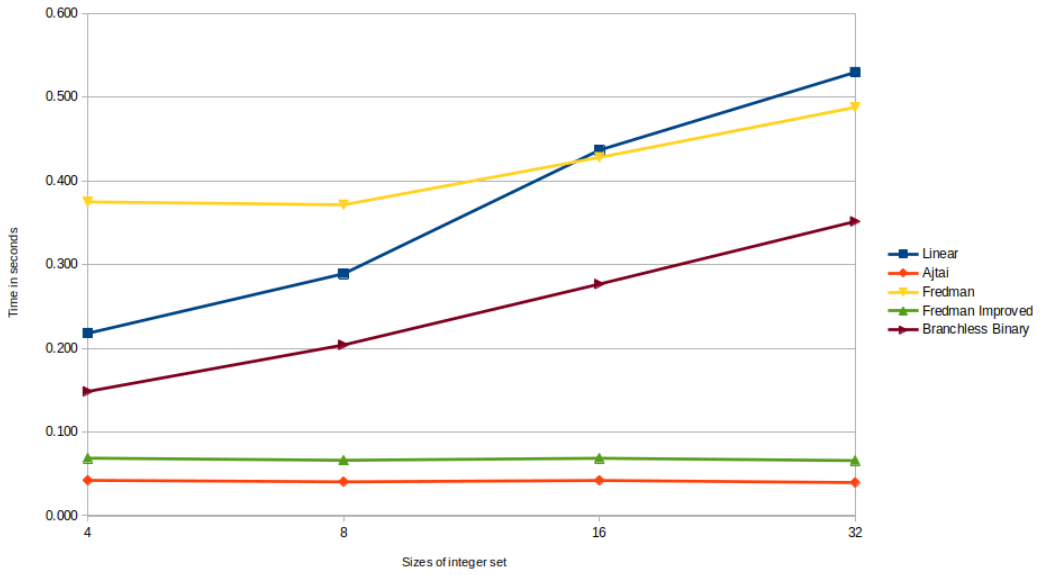

Figure 4.3: Time in seconds for 2^{27} *rank* operations on various implementations

From Figure 4.3, Ajtai method proves to be the fastest, though having the highest memory foot-print. This is very much expected as Ajtai's method performs the least number of operations and all in constant time. This is followed by our improved Fredman approach which surprisingly performs better than branch-less binary search. The direct implementation of Fredman [30] is the slowest, will be due to the possible branch-misprediction that happens when choosing the element with the longest common prefix (between S_i and S_{i+1}).

4.6 Conclusion

In summary, we presented a practical implementation of fast predecessor search on small set on integers by utilising builtin functions and SIMD capabilities of modern processor. Our experiments show that Ajtai's method is the fastest, though with the highest extra memory footprint. This is closely followed by the Improved-Fredman technique we proposed, and the original Fredman's is the slowest. We expect further performance improvements from using higher SIMD registers (such as 512-bits or higher).

It is worth mentioning that Pătraşcu [46] presented a dynamic predecessor search on small set that was implemented by Patrick [22] in their paper *Engineering Predecessor Data Structures for Dynamic Integer Sets*. Their work supported *insertions and deletions* that we did not consider. They also used SIMD instructions similar to ours.

Chapter 5

Dynamic Searchable Prefix Sum

In this chapter, we discussed the problem of dynamic searchable prefix sum with emphasis on good practical performance. We begin by describing the problem with some of its numerous applications. We then cover related works in this area, and then describe our implementation details that are competitive with existing solutions. Finally, we discuss the space requirements, time usage, and open problems.

5.1 Preliminaries

The *Prefix Sum* problem maintains an array A with n integers of *m-bits* each with the following operations supported:

- $sum(i)$ returns $\sum_{j=0}^{i} A[i]$.
- $access(i)$ returns $A[i]$ such that $0 \leq i \leq n$.
- $update(i, \delta)$ modifies $A[i] = A[i] + \delta$ such that $0 \leq A[i] + \delta \leq 2^k - 1$.

A special case of this problem is when $m = 1$, also known as the Dynamic Bit-Vector problem. This maintains a bit-vector of length n that supports *rank, select, and flip/update* operations [52].

Naive approach: Given A of size n, we store A plainly and answer *sum* in $O(n)$ by scanning the array and *update/access* in $O(1)$ with no extra space.

3	6	4	9	14	35	7	21
0	1	2	3	4	5	6	7

Table 5.1: Array A stored plainly to support *increment/decrement* in $O(1)$

3	9	13	22	36	71	79	99
0	1	2	3	4	5	6	7

Table 5.2: Array B such that $B[i] = sum(i)$ to support *sum* in $O(1)$

To support *sum* in $O(1)$, we can use a pre-computed table B of size $O(n \log n)$ such that $B[i] = sum(i)$, but *update* will now be in $O(n)$. Both solutions are not optimal.

Better approach: Using trees, we are able to achieve better results. A Segment-Tree [8] used a balanced binary tree with the internal nodes storing the *sum* of the elements of A while the elements of A are stored in the leaves. With a balanced binary-tree, the height is $\lceil \log n \rceil + 1$, and *sum/update* operations can now be supported in $O(\log n)$ time. A Fenwick-tree [27] improves on the space usage by using an implicit tree to store A using $n+1$ machine words, but still needs $O(\log n)$ to answer the queries. This approach has been extended to b-ary Segment-Tree with height of $\lceil \log_b n \rceil + 1$ allowing for queries to be answered in $O(\log_b n)$ and updates in $O(b \log_b n)$ [21, 36, 50, 52].

5.2 Related work

The foundational work by Fredman and Saks [30] established a lower bound of $\Omega(\log n / \log w)$, which simplifies to $\Omega(\log n / \log \log n)$ for the common case where an integer fits within a word w. They also presented a trade-off $\Omega(logn)$. Yao [57] independently found the same lower

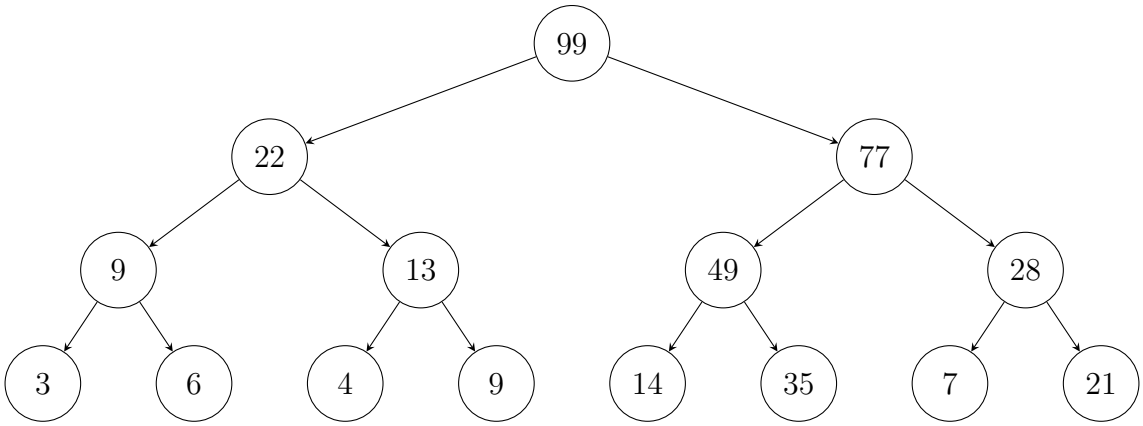

Figure 5.1: Balanced binary tree storing prefix sums of A

bound using a different approach for the amortised complexity of these operations. This indicates that any data structure supporting these operations must incur a logarithmic cost in the worst case. Hampapuram and Fredman [34] demonstrated a $\Omega \log n$ lower bound for amortised complexity of the operations. They show the difficulty in reducing the complexity of these operations without compromising on performance in certain scenarios. Dietz [21] introduced a data structure meeting Fredman and Saks' lower bound for both operations, requiring $\delta = O(\log \log n)$, handling $b = O(\log^{n)}$ elements in constant time, and utilising a Segment-Tree with branching factor b. This solution was enhanced by Raman [52] and Hon [36], though it necessitated large universal tables that scaled poorly with δ and b.

Pătraşcu [50] further refined these lower bounds and trade-offs, presenting a parametric lower bound in terms of w and δ. They distinguished between cases where $= \Omega(w)$ and $\delta = o(w)$. For $\delta = \Omega(w)$, they identified the logarithmic bound $\Omega(\log n)$ and two symmetrical trade-off lower bounds of $\Omega(\log n)$, confirming the optimal results of Segment-Tree and Fenwick-Tree structures. For $= o(w)$, they found a trade-off lower bound of $\Omega(\log n)$, leading to $\Omega(\log n / \log(w/\delta))$. They proposed a data structure achieving $O(\log n / \log(w/\delta))$, optimal for this lower bound. Their approach involved supporting operations in constant time on b elements, building a Segment-Tree with branching factor b, and packing array C in w bits for constant-time manipulation, feasible as long as $b = O(w/(\delta + \log w))$.

Extensive work from theoretical point of view has been studied on this problem [9, 10, 14, 29, 34, 36, 50, 57] due to its promising applications in areas of coding, databases, and creating dynamic data structures. Pibiri [48] explored various c++ implementation of prefix sum data structure and their optimisations.

5.3 Searchable Prefix Sum

The *Searchable Prefix Sum* problem maintains an array A of n positive integers that supports all the operations in the *static prefix sum* with the additional operation below:

- select(i): returns the index of the successor of i in A.

Raman [52] built on [21] to propose a data structure that supports all operations in $O(\log n/\log\log n)$ time while using *kn + o(kn)* bits. They started by solving the problem in small sets of integers that supports all operations in constant time.

Lemma 1: On a RAM model, searchable partial sum problem on a sequence of $m = w^{\epsilon}$ for a fixed ϵ such that $0 \leq \epsilon \leq 1$ on set S and $k = |S|$ and $k \leq w$, the data structure stores S using $O(mw)$ bits and supports operations in $O(1)$ using $O(2^{\epsilon^1 w})$

Proof: Let S be the set of elements that for which their partial sums are to be calculated. Let B be another table such that $B[i] = \sum_{i=0}^{j} A[j]$. Using [21], we can let B to go out of *sync*, where not all updates are immediately effected on B. Otherwise, *updates* will be in $O(m)$. So, for every m updates, we then refresh B to bring it up to date. This brings the *update* operation on B to amortised $O(1)$.

For queries, we maintain a buffer array C on size n to track the out of sync updates. When we want to update $B[i]$ by δ, we instead update $C[i] = C[i] + \delta$. Since $|C[i]| \leq m\delta = w^{O(1)}$, C uses $O(m \log w)$ bits. Dietz [21] supports *sum(i)* by accessing $B[i] + C[i]$ where C is indexed into a pre-computed table.Using *Q-heap* proposed by Fredman [29], Raman showed how to support *search(i)* in $O(1)$.

Theorem 1: For $0 \leq M \leq 2^w$, we can support *insert, delete, update, predecessor and successor* operations in $O(1)$ for $|S| = \log M^{1/4}$. The *successor(x)* is the largest $s \in S$ such that $y \geq x$, while the *predecessor(x)* is the largest $s \in S$ such that $y \leq x$. There is need for a pre-computed table of size $O(M)$ to achieve this.

More information on this can be found in [52].

5.4 Dynamic Searchable Prefix Sum

We propose to implement a variation of Dietz [21] data structure using Single-Instruction-Multiple-Data instructions on A where $|A|$ is a small set of integers. The main concern is how to maintain and update B in amortised $O(1)$, and support *select* in $O(1)$. We came up with simple, yet efficient strategies to maintain B and C using a combination of arrays and SIMD registers using the improved data structure proposed in Chapter 4.5.3 that allows constant time *select*.

Our data structure supports the following operations:

- $sum(i)$: return $\sum_{j=0}^{i} A[i]$
- $increment(i)$: updates $A[i] = A[i] + 1$
- $select(j)$: returns the smallest i such that $sum(i) \geq j$

We support *sum and select* operations in $O(1)$ while *increment* in amortised $O(1)$. Given A of size n, we compute $B[i] = \sum_{j=0}^{i} A[i]$. We maintain C of size $O(w^{O(1)})$ that can be updated in $O(1)$. We determine $m = \max\{B[i] - B[i+1]\}$ to be the largest gap between the elements in B and set a counter to track number of updates so far as *counter*. The idea is to refresh B to bring it up to date with C after every $counter = m$ updates on C. We calculate a mask m_c p of size w that sets p_i to 1 if it is branching bit in B, this will be used to compress values in B. We maintain an array D of size $O(w^{O(1)})$ that stores the compressed values of B such that $D[i] = compress(B[i], m_c)$, and whenever B is refreshed, we reconstruct D. We

should note that C and D can store n items.

Using SIMD registers and builtin instructions, we can store C and D in AVX515 registers[1] allowing up to 512 bits to be processed at the same time. To compress B, we use the standard _pext_u64[2] to extract the *most significant bits* from B in $O(1)$. We know that C and D can support operations such as *addition, subtraction, multiplication, access, e.t.c* in constant time as well.

14	5	6	8	3	1	12	1

Table 5.3: Array A with original sequence

14	19	25	33	36	37	49	50

Table 5.4: Array B with possibly out of *sync* prefix sums

0	0	2	1	1	1	4	3

Table 5.5: SIMD Array C with all the recent updates

14	19	27	37	40	42	58	62

Table 5.6: Array B after $m = 12$ updates is now refreshed with C

5.5 Operations

sum(i): To support $sum(i)$, we simply return $B[i] + \sum C[i]$. This may appear to be slower than normal, but C is expected to fit into very fast memory and can be computed in constant time. So the impact is negligible.

access(i): To support $access(i)$, we simply return $sum(i) - sum(i-1)$.

[1] https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.htmltechs=AVX_512
[2] https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.htmltext=pextig_expand=5088

increment(i): we simply update C by generating an update *mask* starting with all elements in the mask to be zero, and then set $C[i]$ to 1 for $0 \leq i < n$. Adding the *mask* and C is a simple SIMD addition in $O(1)$. We then increment our update counter *counter*. If $counter = m$, we refresh B by setting $B[i] = B[i] + \sum C[i]$ for $0 \leq i < n$. We also compress B into D as described earlier, then recompute m and reset *counter*.

select(j): As D stores the compressed values of B, we can equally compress j. Then search for j in D using SIMD instructions as described in Section 4.5.3. This is also done in $O(1)$.

It is clear that for space usage, we are able to store B using $O(n \log n)$. We only need $O(w^{O(1)}$ to store C and D, which is $o(n)$. This data structure uses $O(n \log n) + o(n)$.

5.6 Optimisations

We can optimise the *access(i)* and *update* operations further. During updates, instead of updating a single element in C, we can generate a mask *mask* that sets $mask[i]$ for all $0 \leq i < n$. Then we set $mask[j] = 1$ for all $i \leq j < n$. This way, C is maintained as a prefix sum as well. When refreshing, we only need to set $B[i] = B[i] + C[i]$. Likewise, when accessing elements or doing sum operations, we can directly access C without having to sum up the values in C.

Chapter 6

Conclusions

Succinct data structures are important component of data mining that allows faster and cheaper processing of voluminous data. In this thesis, we focused on Engineering dynamic data structures for integer compression.

In Chapter 3, we looked into extendible arrays and the effects of internal fragmentation when resizing. We proposed a simple, yet succinct solution to resizing arrays of non-standard *word-size* width using a variety of factors. Our implementation allows for *append-only* extendible arrays that are more space efficient and competitive in speed with existing solutions. Building on this, we built an implementation for *width=1* to have an *append-only* dynamic bitvector. We also implemented an *append-only* dynamic Elias-Fano and compare this with existing solutions. Our implementation works well even if we do not know the *universe* before hand. However, it will be good to investigate how to balance the number of rebuilds vs resize factors empirically even though the theoretical cost is amortised $O(1)$.

In Chapter 4, we looked at the problem of fast predecessor search in small sets. This is a well-know problem with little practical implementation. As far as we are aware, we implemented the first constant time fast predecessor search in static sets. Using SIMD and builtin instructions available in most processors now, we were able to practically implement

a data structure that was otherwise considered impractical. We also provided an improved version of the Fredman's [29] by compressing the input in $O(1)$ to speed up the rank-operation which proves to be faster. We also investigated the problem of reducing the number of branching bits by shifting the numbers in the set with a constant value x which is very promising. As open problem, can we determine the optimal x to get the least branching bits in a set? It will also be good to extend the experiments on processors with larger than 256-bits SIMD capacity.

In Chapter 5, we investigated the well-known problem of dynamic searchable prefix sum. Building on the data structures in earlier chapters, the proposed and implemented a dynamic searchable prefix sum with increment and select. SIMD instructions allow us to achieve some of the presented lower-bounds and assumptions that were earlier on impractical. As future work, we intend to implement these data structures to measure and assess empirical performance.

Bibliography

[1] Advanced Micro Devices Inc. *AMD64 Architecture – Programmer's Manual Volume 3: General-Purpose and System Instructions*, September 2020. URL `https://www.amd.com/system/files/TechDocs/24594.pdf`.

[2] M. Ajtai, M. Fredman, and J. Komlós. Hash functions for priority queues. *Information and Control*, 63(3):217 – 225, 1984. ISSN 0019-9958. doi: https://doi.org/10.1016/S0019-9958(84)80015-7. URL `http://www.sciencedirect.com/science/article/pii/S0019995884800157`.

[3] V. N. Anh and A. Moffat. Impact transformation: Effective and efficient web retrieval. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 3–10, Tampere, Finland, 2002.

[4] V. N. Anh and A. Moffat. Inverted index compression using word-aligned binary codes. *International Journal of Information Retrieval*, 8(1):151–166, 2005.

[5] Arm Limited. *A64 Instruction Set Reference*, 2018. URL `https://developer.arm.com/documentation/100076/0100/a64-instruction-set-reference`.

[6] D. Arroyuelo, P. Davoodi, and S. R. Satti. Succinct dynamic cardinal trees. *Algorithmica*, 74:742–777, 2016.

[7] Jon Bentley. *Programming Pearls (2nd Ed.)*. ACM Press/Addison-Wesley Publishing Co., USA, 1999. ISBN 0201657880.

[8] Jon Louis Bentley. Solutions to klee's rectangle problems. Unpublished manuscript, 1977.

[9] Philip Bille, Anders Roy Christiansen, Nicola Prezza, and Frederik Rye Skjoldjensen. Succinct partial sums and fenwick trees. In *International Symposium on String Processing and Information Retrieval*, pages 91–96. Springer, 2017.

[10] Philip Bille, Anders Roy Christiansen, Patrick Hagge Cording, Inge Li Gørtz, Frederik Rye Skjoldjensen, Hjalte Wedel Vildhøj, and Søren Vind. Dynamic relative compression, dynamic partial sums, and substring concatenation. *Algorithmica*, 80(11): 3207–3224, 2018.

[11] A. Brodnik, S. Carlsson, E. Demaine, J. Ian Munro, and R. Sedgewick. Resizable arrays in optimal time and space. In *Lecture Notes in Computer Science*, pages 37–48. 1999.

[12] Andrej Brodnik. Computation of the least significant set bit. In *Proceedings of the 2nd Electrotechnical and Computer Science Conference*, volume 90, Portoroz, Slovenia, 1993.

[13] Stefan Büttcher, Charles Clarke, and Gordon Cormack. *Information retrieval: implementing and evaluating search engines*. MIT Press, 2010.

[14] Timothy M. Chan and Mihai Pătraşcu. Counting inversions, offline orthogonal range counting, and related problems. In *Proceedings of the twenty-first annual ACM-SIAM symposium on Discrete Algorithms*, pages 161–173. SIAM, 2010.

[15] David Clark. *Compact Pat trees*. PhD thesis, University of Waterloo, 1998.

[16] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, Cambridge, MA, 4th edition, 2022.

[17] Intel Corporation. Intel intrinsics guide. `https://www.intel.com/content/www/us/en/docs/intrin`
439. $Accessed : 1 July 2024$.

[18] Tom Coughlin. 175 zettabytes by 2025. *Forbes*, 2018. URL `https://www.forbes.com/sites/tomcoughlin/2018/11/27/175-zettabytes-by-2025/`. Accessed: 1 July 2024.

[19] Mikael Degermark, Andrej Brodnik, Svante Carlsson, and Stephen Pink. Small forwarding tables for fast routing lookups. In *ACM SIGCOMM Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*, pages 3–14. ACM, 1997. doi: 10.1145/263105.263133.

[20] O. Delpratt. *Space efficient in-memory representation of XML documents*. PhD thesis, University of Leicester, 2009.

[21] Paul F. Dietz. Optimal algorithms for list indexing and subset rank. In *Workshop on Algorithms and Data Structures*, pages 39–46. Springer, 1989.

[22] Patrick Dinklage, Johannes Fischer, and Alexander Herlez. Engineering predecessor data structures for dynamic integer sets. *CoRR*, abs/2104.06740, 2021. URL `https://arxiv.org/abs/2104.06740`.

[23] R. W. Doran. *Computer Architecture: A structured approach*. Academic Press, London, 1979.

[24] P. Elias. Efficient storage and retrieval by content and address of static files. *Journal of the ACM (JACM)*, April 1974.

[25] Peter Elias. Universal codeword sets and representations of the integers. *IEEE Transactions on Information Theory*, 21(2), 1975.

[26] Robert Mario Fano. On the number of bits required to implement an associative memory. Technical Report Memorandum 61, Computer Structures Group, MIT, 1971.

[27] Peter M. Fenwick. A new data structure for cumulative frequency tables. *Software: Practice and Experience*, 24(3):327–336, 1994.

[28] Paolo Ferragina and Roberto Grossi. The string b-tree: A new data structure for string search in external memory and its applications. *J. ACM*, 46 (2):236–280, March 1999. ISSN 0004-5411. doi: 10.1145/301970.301973. URL `https://doi.org/10.1145/301970.301973`.

[29] Michael L. Fredman. The complexity of maintaining an array and computing its partial sums. *Journal of the ACM*, 29(1):250–260, January 1982.

[30] Michael L. Fredman and Dan E. Willard. Surpassing the information theoretic bound with fusion trees. *Journal of Computer and System Sciences*, 47(3):424 – 436, 1993. ISSN 0022-0000. doi: https://doi.org/10.1016/0022-0000(93)90040-4. URL `http://www.sciencedirect.com/science/article/pii/0022000093900404`.

[31] Simon Gog et al. From theory to practice: Plug and play with succinct data structures. In *International Symposium on Experimental Algorithms*. Springer, 2014.

[32] Rodrigo González, Szymon Grabowski, Veli Mäkinen, and Gonzalo Navarro. Practical implementation of rank and select queries. In *Workshop on Experimental Algorithms (WEA)*, pages 27–38, 2005.

[33] R. Grossi and G. Ottaviano. The wavelet trie. In *Proceedings of the 31st symposium on Principles of Database Systems - PODS '12*, 2012.

[34] Haripriyan Hampapuram and Michael L. Fredman. Optimal biweighted binary trees and the complexity of maintaining partial sums. *SIAM Journal on Computing*, 28(1): 1–9, 1998.

[35] Yedidya Hilewitz. *Advanced bit manipulation instructions: Architecture, implementation and applications*. PhD thesis, Princeton University, United States – New Jersey, 2008.

[36] Wing-Kai Hon, Kunihiko Sadakane, and Wing-Kin Sung. Succinct data structures for searchable partial sums with optimal worst-case performance. *Theoretical Computer Science*, 412(39):5176–5186, 2011.

[37] Intel Corporation. *Intel (R) 64 and IA-32 Architectures – Software Developer's Manual – Volume 2: Instruction Set Reference, A-Z*, September 2016. URL `https://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-archit`

[38] G. Jacobson. *Succinct static data structures*. PhD thesis, Carnegie Mellon University, 1989.

[39] Stelios Joannou and Rajeev Raman. An empirical evaluation of extendible arrays. In *Lecture Notes in Computer Science*, volume 6630, pages 447–458, 2011. doi: 10.1007/978-3-642-20662-$7_3$8.

[40] Tomasz Jurkiewicz and Kurt Mehlhorn. On a model of virtual address translation. *ACM Journal of Experimental Algorithmics*, 19:1–28, 2015. doi: 10.1145/2656337.

[41] Christopher Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[42] J. I. Munro, V. Raman, and A. J. Storm. Representing dynamic binary trees succinctly. In *SODA*, pages 529–536, 2001.

[43] G. Navarro and E. Providel. Fast, small, simple rank/select on bitmaps. In *Experimental Algorithms*, pages 295–306. 2012.

[44] Gonzalo Navarro. *Compact Data Structures: A practical approach*. University of Cambridge, New York, NY, 2016.

[45] Mihai Patrascu and Mikkel Thorup. Dynamic integer sets with optimal rank, select, and predecessor search. *CoRR*, abs/1408.3045, 2014. URL `http://arxiv.org/abs/1408.3045`.

[46] Mihai Patrascu and Mikkel Thorup. Dynamic integer sets with optimal rank, select, and predecessor search. *CoRR*, abs/1408.3045, 2014. URL `http://arxiv.org/abs/1408.3045`.

[47] G. E. Pibiri and R. Venturini. Dynamic elias-fano representation. In *28th Annual Symposium on Combinatorial Pattern Matching (CPM 2017)*, 2017.

[48] Giulio Ermanno Pibiri and Rossano Venturini. Practical trade-offs for the prefix-sum problem. *Software: Practice and Experience*, 51(5):921–949, October 2020. ISSN 1097-024X. doi: 10.1002/spe.2918. URL `http://dx.doi.org/10.1002/spe.2918`.

[49] Nicola Prezza. A framework of dynamic data structures for string processing. In Costas S. Iliopoulos, Solon P. Pissis, Simon J. Puglisi, and Rajeev Raman, editors, *16th International Symposium on Experimental Algorithms, SEA 2017, June 21-23, 2017, London, UK*, volume 75 of *LIPIcs*, pages 11:1–11:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017. doi: 10.4230/LIPIcs.SEA.2017.11. URL `https://doi.org/10.4230/LIPIcs.SEA.2017.11`.

[50] Mihai Pătraşcu and Erik D. Demaine. Logarithmic lower bounds in the cell-probe model. *SIAM Journal on Computing*, 35(4):932–963, 2006.

[51] R. Raman and S. Rao. Succinct dynamic dictionaries and trees. In *Automata, Languages and Programming*, pages 357–368. 2003.

[52] R. Raman, V. Raman, and S. Rao. Succinct dynamic data structures. In *Lecture Notes in Computer Science*, pages 426–437. 2001.

[53] Robert Rice and J. Plaunt. Adaptive variable-length coding for efficient compression of spacecraft television data. *IEEE Transactions on Communications*, 16(9):889–897, 1971.

[54] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(4):623–656, 1948. doi: 10.1002/j.1538-7305.1948.tb00917.x.

[55] Larry H. Thiel and H. S. Heaps. Program design for retrospective searches on large data bases. *Information Storage and Retrieval (ISR)*, 8(1):1–20, 1972.

[56] S. Vigna. Broadword implementation of rank/select queries. In *Experimental Algorithms*, pages 154–168. 2008.

[57] Andrew C. Yao. On the complexity of maintaining partial sums. *SIAM Journal on Computing*, 14(2):277–288, 1985.

[58] D. Zhou, D. Andersen, and M. Kaminsky. Space-efficient, high-performance rank and select structures on uncompressed bit sequences. In *Experimental Algorithms*, pages 151–163. 2013.

[59] Justin Zobel and Alistair Moffat. Inverted files for text search engines. *ACM Comput. Surv.*, 38(2):6–es, jul 2006. ISSN 0360-0300. doi: 10.1145/1132956.1132959. URL `https://doi.org/10.1145/1132956.1132959`.